

Portfolio GDD / Combat Design

阴阳之力

Boss 战斗 Demo

Unreal Engine 5.7

C++ / Blueprint

GAS

Behavior Tree

DOCUMENT MAP

- 01 项目概况
- 02 游戏体验
- 03 3C
- 04 战斗设计
- 05 伤害结算与反馈
- 06 AI 设计

目录

01 项目概况.....	5
1.1 项目名片与制作规格.....	5
1.2 项目特点总览.....	6
1.3 技术栈.....	6
02 游戏体验.....	8
2.1 核心体验.....	8
2.2 体验设计支柱.....	8
2.3 战斗逻辑.....	10
2.4 战斗循环.....	10
2.5 压力曲线.....	11
2.6 压力曲线说明.....	11
03 3C.....	12
3.1 角色.....	12
3.2 沉寂：角色概念图 + 刀与刀鞘.....	13
3.3 裴兴：角色概念图 + 朴刀.....	14
3.4 相机.....	15
3.4.1 Camera 枢轴、镜头运动与构图.....	17
3.4.2 实机镜头演示.....	18
3.4.3 镜头优先级与高优先级触发.....	19
3.4.4 Camera 遮挡淡化机制.....	21
3.4.5 Camera 参数与调参入口.....	22
3.5 输入.....	24
3.5.1 输入状态总流程.....	25

3.5.2 Dodge 与 PDA 详细流程	27
3.5.3 状态路由表	31
04 战斗设计	32
4.1 呼吸系统	32
4.2 完美操作	37
4.2.1 完美操作的游戏循环	39
4.3 阳之力	42
4.4 禅意时刻	43
4.5 呼吸值, 阳之力 UI 展示	45
4.5.1 呼吸值 UI 展示图占位	45
4.5.2 呼吸值 UI 分层和优先级	46
4.5.3 动态变化与 PreZen Bar 机制	47
4.6 阳之力	50
4.6.1 阳之力 UI 展示图占位	50
4.6.2 阳之力 UI 分层和优先级	50
4.6.3 获取、使用与受击动态	51
05 伤害链条与伤害反馈	53
5.1 伤害判定与反馈开始的流程图	53
5.2 伤害流程的代码框架	56
5.3 伤害计算公式	58
5.4 受击反馈类型与展示	61
06 AI 设计	65
6.1 敌人设计思路	65
6.2 设计目标	66

6.3 战斗难点.....	67
6.4 难度平衡.....	67
6.5 特殊机制和实现方案.....	69
6.5.1 AttackIntent 显式调度.....	69
6.5.2 CombatPressure	69
6.5.3 距离和攻击范围兜底.....	70
6.5.4 防御、反击与受击恢复.....	71
6.5.5 阶段与冷却	71
6.6 BT 结构.....	72

01 项目概况

1.1 项目名片与制作规格

项目名	阴阳之力 Boss 战斗 Demo
类型定位	第三人称动作 Boss 战斗原型；面向作品集投递的核心战斗体验。
引擎版本	Unreal Engine 5.7
核心框架	C++ / Blueprint / Gameplay Ability System
制作规格	约 2 分钟核心 Boss 战；以战斗系统、3C , AI 和打击反馈感为展示重点。
人数 / 周期	1 人独立制作；2026.03 - 2026.05；约 10 周，累计约 229 小时。
核心特点	以 BreathValue 统一战斗节奏、3C 反馈与 Boss 响应；完美操作与 Zen Moment 构成反制闭环。

1.2 项目特点总览

顺序	重点	设计含义	后续展开
01	核心命题	专注于当下并冷静应对危险才是取胜之道。	游戏体验 / 核心机制
02	战斗循环	玩家行动加剧呼吸值，呼吸状态引导玩家改变操作	游戏体验 / AI
03	3C	角色、控制、相机共同服务“高压与平静”的体验曲线。	3C 设计与反馈设计
04	数值闭环	呼吸、疲劳、完美收益、禅境状态与伤害结构形成可调参模型。	数值模型
05	技术落地	C++ / GAS / DataAsset 承载核心逻辑，Blueprint、UMG 与 Niagara 承接表现。	技术结构

1.3 技术栈

模块	框架 / 插件	在本项目中的用途
战斗能力	GAS、Gameplay Tags、Gameplay Tasks	属性、GE、Tag、Ability 与战斗状态流转。

输入与控制	Enhanced Input、InputCore、WinDualShock	键鼠输入映射。
动作与位移	Motion Matching (Pose Search) 、 Motion Trajectory、Animation Locomotion、Motion Warping	运动匹配、轨迹预测、转身/接近/命中位移修正。
AI 决策	BT / Blackboard、EQS、AI Module	Boss 状态、距离判断、目标点刷新与攻击选择。
反馈与 UI	Niagara、UMG、Slate / SlateCore	命中、受伤、边框、调试与去 UI 化反馈表现。

02 游戏体验

2.1 核心体验

高压、专注与反制

玩家在战斗中会遭受连续不断的高压攻击态势。为了应对这种高压态势，玩家会将注意力集中于一瞬间并应用于“完美系统”，为自己获得瞬间的攻击窗口并一举扭转战斗主动权。其中“完美闪避”可以让玩家在化解敌人进攻的同时保持自己的攻击姿态，“完美格挡”可以在恢复大量生命值和阳之力的同时为自己获得更多的喘息窗口。最终在连续不断的专注力中，玩家达到“禅意”境界并掌控时间与空间。

2.2 体验设计支柱

序号	支柱	设计含义
01	呼吸系统	用 BreathValue 表示压力，驱动节奏、风险与反制窗口。
02	完美系统	精准闪避/格挡换取恢复、攻击窗口与主动权。
03	禅意系统	连续专注触发 Zen，获得短暂而强力的反制爆发。
04	数据驱动的动态化 AI 行为逻辑	根据玩家行为与呼吸状态调整敌人行动。

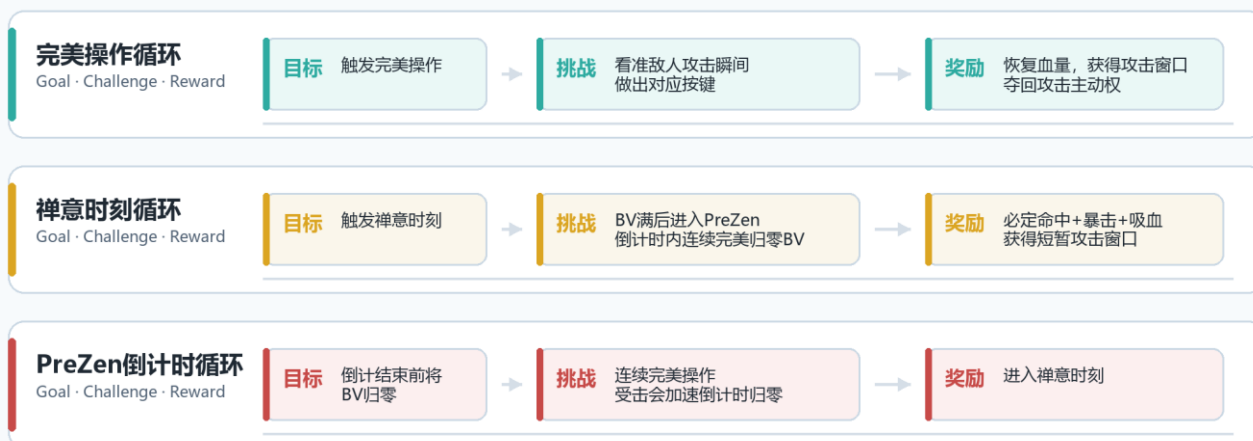
2.3 战斗逻辑

观察敌人意图与自身情况 -> 依据自身情况对敌人的攻击做出瞬间判断 -> 呼吸值改变 -> 观察呼吸值并部署下一步动向 -> 敌人的行为发生动态变化 -> 重新观察敌人意图与自身情况。



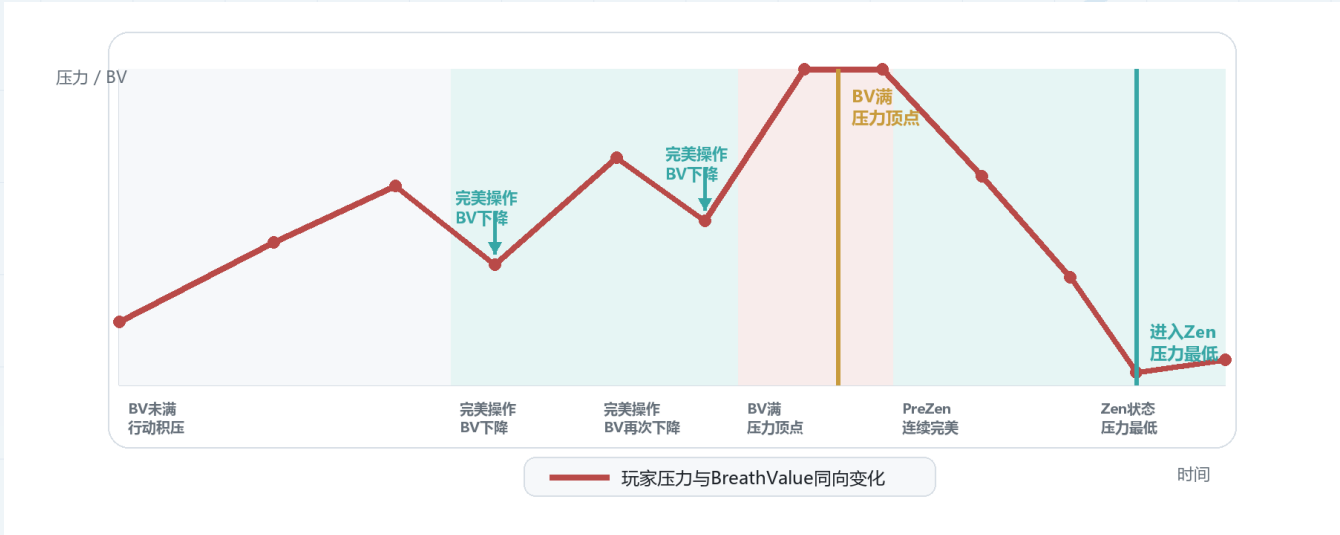
敌人的动态变化会重新回到玩家观察与判断，形成持续高压下的决策闭环。

2.4 战斗循环



三个小标题各自形成目标、挑战、奖励闭环，再由 BV / PreZen / Zen 串联为完整战斗体验。

2.5 压力曲线



2.6 压力曲线说明

前期呼吸值未满	玩家 BV 值提高，代表玩家压力增加；通过完美操作让 BV 降低，玩家压力随之减少。
呼吸值满时	BV 到达顶点，压力也到达顶点；通过连续完美操作进入 Zen 状态，BV 降到最低点，压力也降到最低。

03 3C

3.1 角色

角色	战斗定位	视觉关键词	武器图要点
沈寂	玩家主角	冷静、克制、黑青布甲、 阴阳圆肩甲、轻装机动	佩刀与刀鞘成套出现；刀身 修长，适合快速拔刀、收 刀。
裴兴	Boss	压迫、黑金红、兽面肩 甲、破损披挂、重量感	朴刀为长柄重刃；强调横 扫、挑斩、压迫距离与霸体 攻击的轮廓。

3.2 沈寂：角色概念图 + 刀与刀鞘



图 3C-1 沈寂角色概念与武器图：佩刀、刀鞘并列展示

3.3 裴兴：角色概念图 + 朴刀



3.4 相机

当前战斗镜头使用是“战斗枢轴 Pivot + 距离构图 + 状态响应 + CameraShot 高优先级 + 遮挡物淡化”的组合求解。

CameraV2 战斗镜头主流程

入口: UpdateCameraSystem -> bUseCombatCameraV2 -> UpdateCameraCombatV2 -> CameraComponent.



图 3C-4 Camera 战斗镜头主流程与各层驱动数据

3.4.1 Camera 枢轴、镜头运动与构图

战斗镜头先求 AimPoint，再用 CombatCameraV2Pivot 作为滞后的运动锚点。玩家快速移动、闪避或越界时，Pivot 会被 MaxPivotLag、BoundaryHold 和 PivotPlaneGuard 限制在合法范围内；Camera 则从 Pivot 反向退开并叠加侧向、高度与 FOV，形成最终画面构图。

Camera 枢轴 / 镜头运动与 FOV 构图

Pivot 是滞后的运动锚点；Camera 由 Pivot、AimPoint、Arm、Side、Height 推导真实机位，并用 FOV 形成战斗构图。

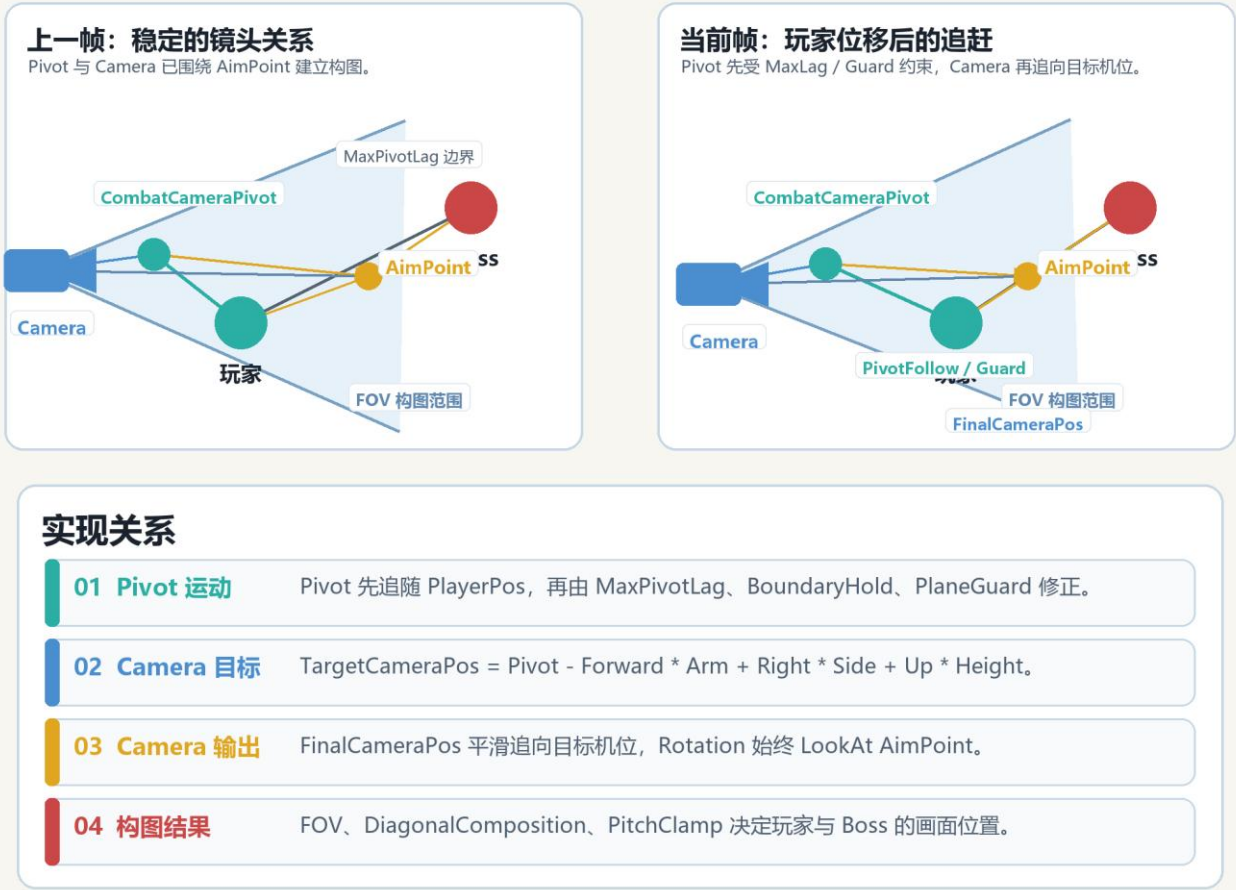
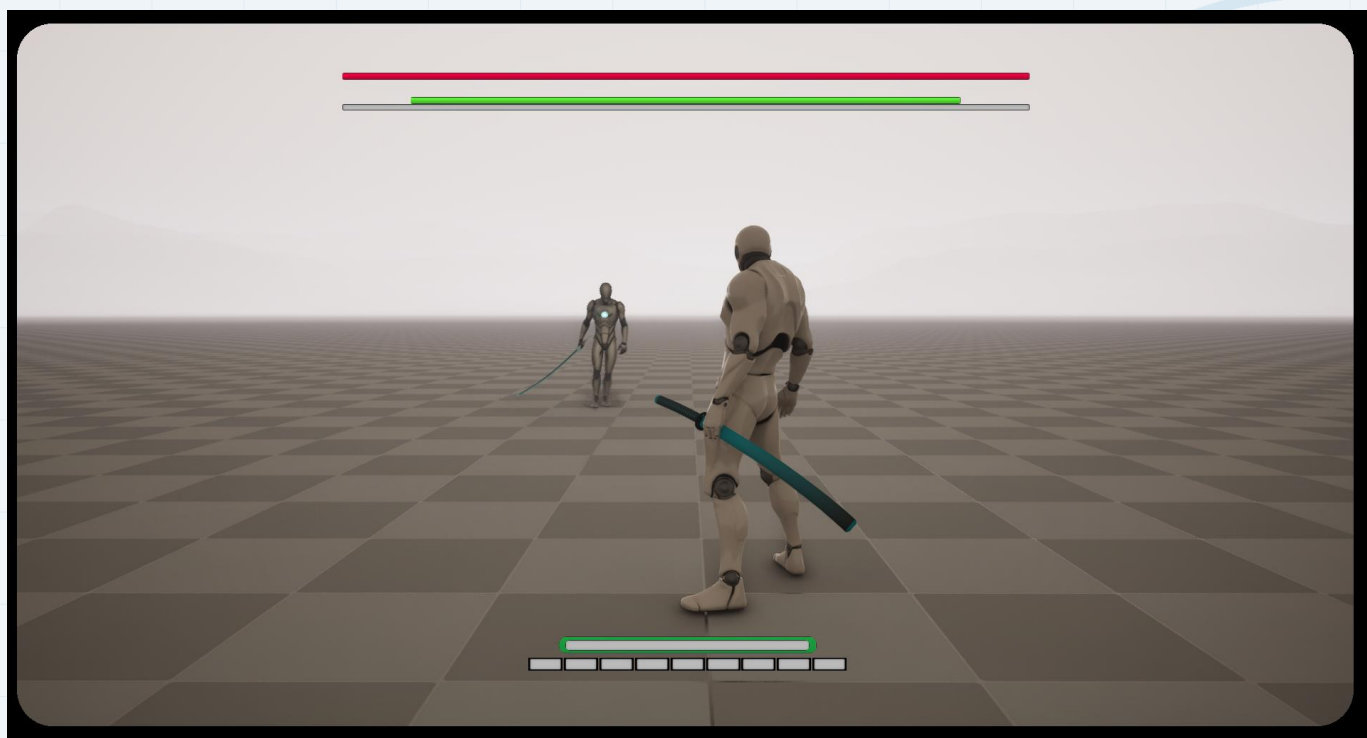
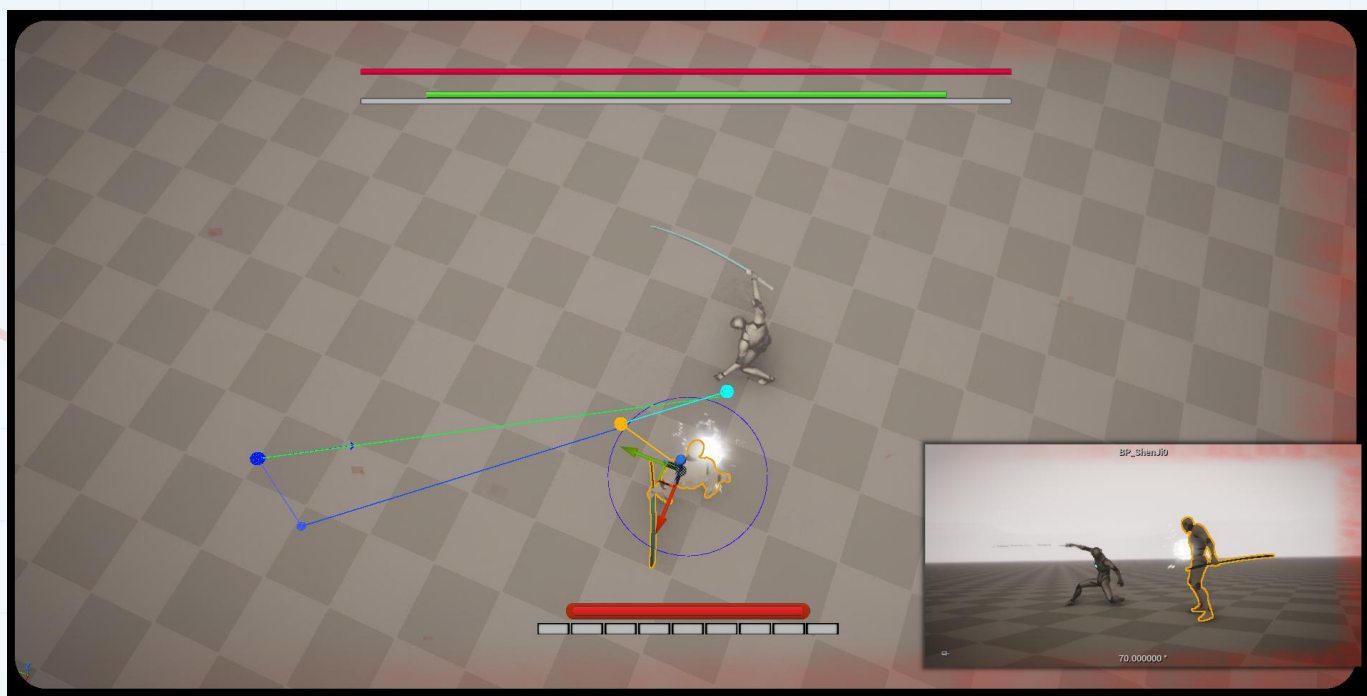


图 3C-5 Camera 枢轴与镜头运动前后、FOV 构图范围

3.4.2 实机镜头演示



镜头系统实机镜头演示



实机镜头 debug 演示

3.4.3 镜头优先级与高优先级触发

镜头的优先级分成“镜头拥有者”和“输出安全约束”。CameraShot 会短时覆盖基础战斗镜头；命中反馈以加性方式叠加；Fast、Dodge 和枢轴越界改变响应速度；遮挡淡化不抢镜头，只保证最终可读性。

Camera 优先级与触发条件

高优先级主要发生在 CameraShot、加性反馈和状态驱动的枢轴响应；遮挡淡化是最终输出安全层。



读法：上层覆盖下层；S0 是最终安全约束，不参与抢镜头，但会修正遮挡可读性。

图 3C-6 CameraV2 优先级栈与触发条件

优先级	触发条件	驱动数据	覆盖/退出规则
S0 输出安全	世界几何体挡住玩家、Boss 或屏幕战斗窗口。	UpdateCombatOccluderFade / SphereSweep / DynamicMaterial	不拥有镜头；在最终输出后淡化遮挡物。
P1 CameraShot	ANS_PerfectActionCameraShot 激活，阶段为 PDACombo、PDAFinisher、PerfectParryAttack、ParryFinisher、ZenAttack。	FPerfectActionCameraShotTuning: Anchor、Offset、LookAt、FOV、曲线、Blend	混合覆盖相机 Pos/Rot/FOV；可锁定起始构图。
P2 加性反馈	命中、受击、完美操作反馈、FOVShake。	ActiveAdditives / AddCameraImpulse / ConsumeCameraAdditives	叠加偏移和旋转，保留 V2 主镜头。
P3 状态响应	Fast、Dodge、PerfectDodgeFeedback、枢轴越界。	SpeedAlpha / DodgeAlpha / BoundaryUrgency / Urgency	改变 Pivot MaxLag、跟随速度、位置和旋转响应。
P4 战斗主镜头	SoftLockOnTarget 有效且 bUseCombatCameraV2=true。	DistanceAlpha / CombatCameraV2Pivot / AimPoint / Arm / Height / Side / FOV	持续拥有战斗镜头；输出给 CameraComponent。
P5 自由镜头	未锁定目标或战斗退出。	FreeCameraData / 鼠标输入 / 非战斗移动	低优先级探索镜头；锁定后让位给 V2。

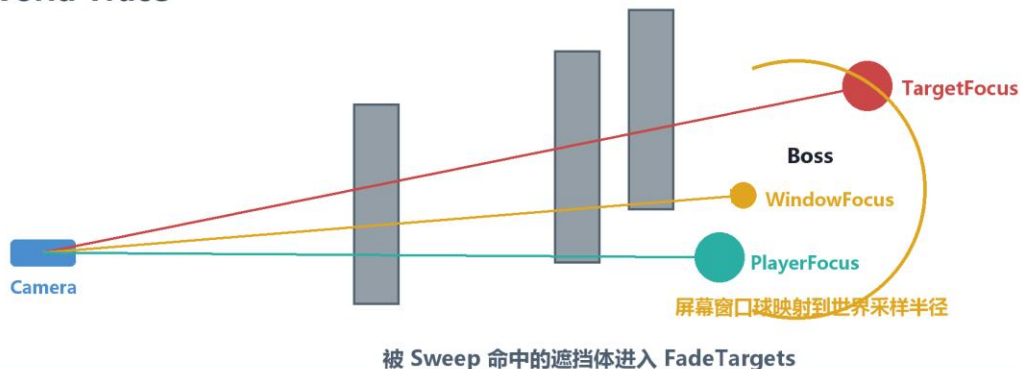
3.4.4 Camera 遮挡淡化机制

遮挡处理现在以世界几何淡化为主：CameraV2 完成位置、旋转和 FOV 输出，UpdateCombatOccluderFade 会从相机向玩家焦点、Boss 焦点以及屏幕战斗窗口做球扫；命中的墙体、柱体等会被替换为动态材质并驱动透明度，离开遮挡后恢复原材质。

Camera 遮挡淡化机制

主动使用世界几何遮挡淡化：让挡住玩家、Boss 或屏幕战斗窗口的墙柱短暂透明。

World Trace



Runtime Fade

- 1 **BuildCombatOccluderFadeWindow**
玩家与 Boss 投影到屏幕，得到中心、半径、UV 和权重。
- 2 **GatherCombatOccluderFadeTargets**
Camera 到 Player / Boss / Window 做 ECC_Camera 球扫。
- 3 **ShouldFadeCombatOccluderComponent**
忽略 Pawn、目标、自身、不可见体和 InstancedStaticMesh。
- 4 **ApplyCombatOccluderFadeEntry**
创建动态材质，写入 Opacity / FadeAlpha / Window 参数。
- 5 **RestoreCombatOccluderFadeEntry**
离开遮挡后按 FadeOutSpeed 恢复原材质。

关键参数

TraceRadius = 18.f	TargetOpacity = 0.25f	FadeIn = 10.f	FadeOut = 7.f
WindowPadding = 0.08f	Min/MaxRadius = 0.16f / 0.48f	Softness = 0.06f	

读法：先收集遮挡目标，再按淡入/淡出动态写材质参数，保障战斗镜头可读性。

图 3C-7 CameraV2 世界遮挡淡化：球扫、屏幕窗口与动态材质



实机演示：CameraV2 世界遮挡淡化：球扫、屏幕窗口与动态材质

3.4.5 Camera 参数与调参入口

主要调参入口是 CameraModeDataAsset。角色类保存运行态数据，例如 CombatCameraV2Pivot、CombatCameraV2AimPoint、CombatCameraV2Urgency、CombatCameraV2DodgeAlpha 和遮挡淡化条目；DataAsset 则决定每一层的默认参数和响应速度。

CameraV2 数据资产参数分组

主要调参入口仍是 CameraModeDataAsset; 角色侧以 bUseCombatCameraV2=true 选择 V2 求解器。

00 基础距离

Min/MaxDist 200.f-1500.f
Arm 300.f-800.f
Height 50.f-200.f
FOV 82.f-90.f

01 目标稳定

Near/FarBias 0.7f/0.3f
MidpointLookTarget
MinStableLookDir 80.f

03 侧向构图

DynamicLateral=true
DeadZone=35.f
RightLimit 120.f/70.f

09 Pivot

Calm/Fast/Dodge MaxLag
200.f/85.f/95.f
Follow=6.f

09 Position/Rotation

Pos 10.f/22.f
Yaw 11.f/18.f
Pitch 9.f/14.f

09 Diagonal

Enable=true
Strength=0.45f
Start/Full 180.f/420.f

04 OcclusionFade

Enable=true
Trace=18.f
Opacity=0.25f
Fade 10.f/7.f

07 CameraShot

PDA Combo / PDA Finisher
PerfectParryAttack
ParryFinisher / ZenAttack
AnimNotify 触发硬特写

实现来源: YinYangCombatCameraV2.cpp、CameraModeDataAsset.h、ANS_PerfectActionCameraShot.cpp、UpdateCombatOccluderFade。

图 3C-8 CameraV2 数据资产参数分组与默认值

参数组	字段	当前默认	设计用途
基础距离	MinDist / MaxDist	200.f / 1500.f	将玩家-Boss 距离归一化为 DistanceAlpha。
臂长	MinArm / MaxArm / MidRangeArmBoost	300.f / 800.f / 90.f	控制远近镜头和中距离拉开感。
高度与 FOV	NearHeight/FarHeight, NearFOV/FarFOV	50.f/200.f, 82.f/90.f	随距离改变俯仰观看空间和视野宽度。
Pivot	Calm/Fast/Dodge MaxLag	200.f/85.f/95.f	决定玩家可在镜头枢轴前后带出的最大距离。
PivotGuard	MaxForwardLag / MinSideLag	0.f / 35.f	限制枢轴不得漂向 Boss 方向, 并保留侧向量。

响应	Position / Yaw / Pitch Response	10.f, 11.f, 9.f	普通状态下相机位置和旋转追赶速度。
急迫响应	Urgent Position / Yaw / Pitch	22.f, 18.f, 14.f	速度、边界压力或闪避使 Urgency 上升时使用。
遮挡淡化	TraceRadius / Opacity / Fade	18.f / 0.25f / 10.f-7.f	控制世界遮挡体检测与透明恢复。

3.5 输入

当前输入以键鼠为主。左键是核心攻击输入，会根据角色当前状态被路由到不同动作；F 直接负责格挡/完美格挡，不需要先进入右键防御。

输入	行动	状态说明
WASD	移动	角色移动输入；战斗中也参与闪避方向和快移方向判断。
鼠标移动	控制摄像头	自由镜头下控制视角；战斗镜头中主要提供视角感知。
鼠标左键	Attack	常态为普通攻击；在窗口中路由到 DodgeAttack、PDA Combo、ParryAttack、Sprint/Fast Attack。
鼠标右键	防御	进入防御姿态；不再作为 Perfect Parry 的前置条件。
L_Shift	Sprint	按住进入 Sprint；Sprint 中按 Space 且继续按住 L_Shift 时进入 Fast。
Space	Dodge / Perfect Dodge / Fast	常态触发 Dodge；完美闪避窗口触发 Perfect Dodge；Sprint+Space+持续 L_Shift 进入 Fast。
F	格挡 / Perfect Parry	直接格挡输入；在完美格挡时机触发 Perfect Parry，无需先按右键防御。
E	回血	触发治疗行为。
R	Finisher	PDA Combo 01-04 任意招式中可触发 PerfectDodgeAttackFinisher。

3.5.1 输入状态总流程

下图更新了 Fast 的触发条件：Fast 必须由 Sprint 状态下按 Dodge，并且持续按住 L_Shift 进入。格挡反击也改为 F 直接触发。



输入状态总流程 B

格挡反击、右键防御与 Dodge 入口；详细 Dodge / PDA 链路见下一张图。

格挡与反击：F 直接触发

完美格挡窗口

敌方攻击即将命中；无需先按右键防御。

Perfect Parry

窗口内按 F。

ParryAttack

之后按 LMB 反击。

RMB：防御

独立防御；不是 F 格挡前置。

Dodge 入口提示

Space：Dodge

普通闪避；可接追击。

完美窗口内 Space

进入 Perfect Dodge；PDA 见下图。

链路重点

Dodge / PDA 见下图。

图 3C-9B 输入状态总流程：格挡反击与 Dodge 入口

3.5.2 Dodge 与 PDA 详细流程

Dodge 分成普通闪避追击和 Perfect Dodge Attack 两条链路。

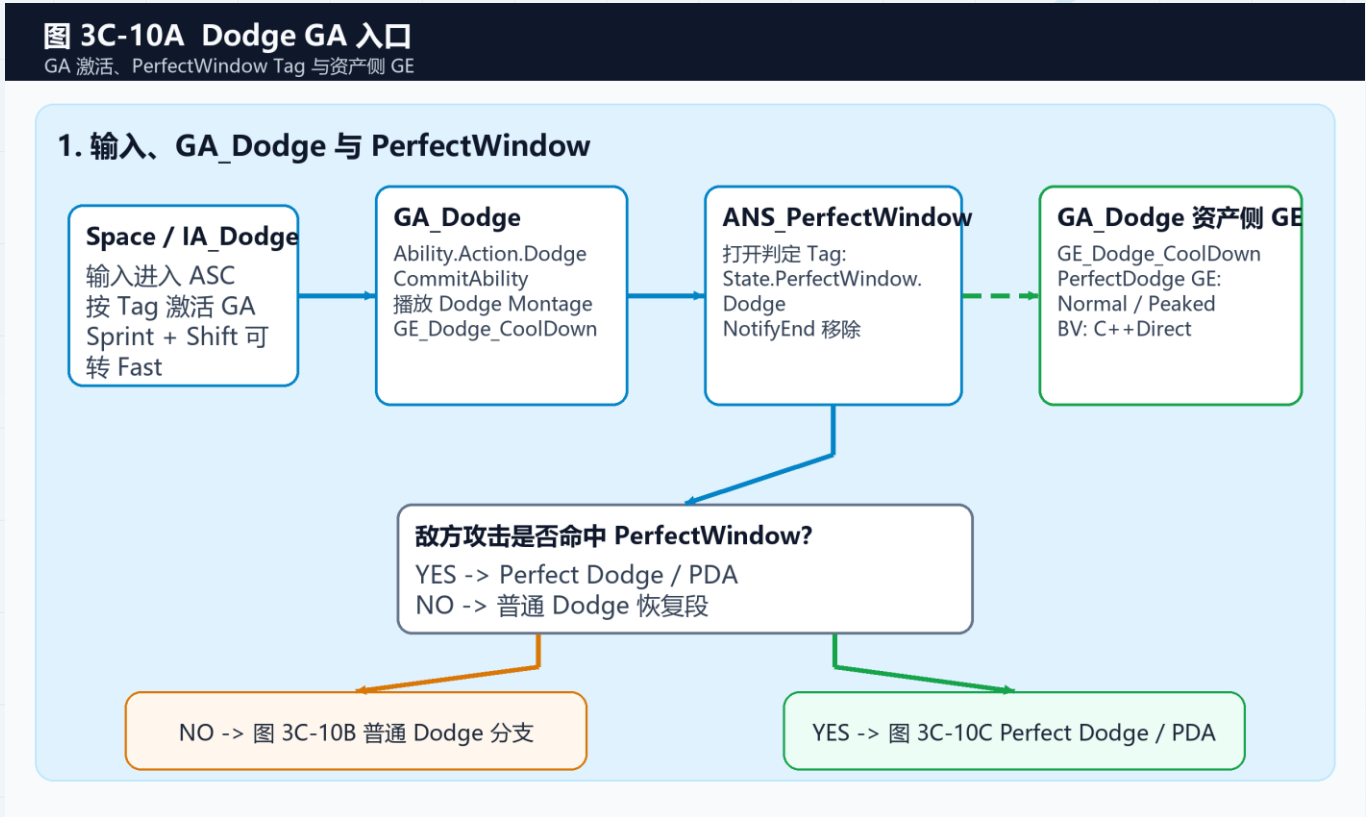
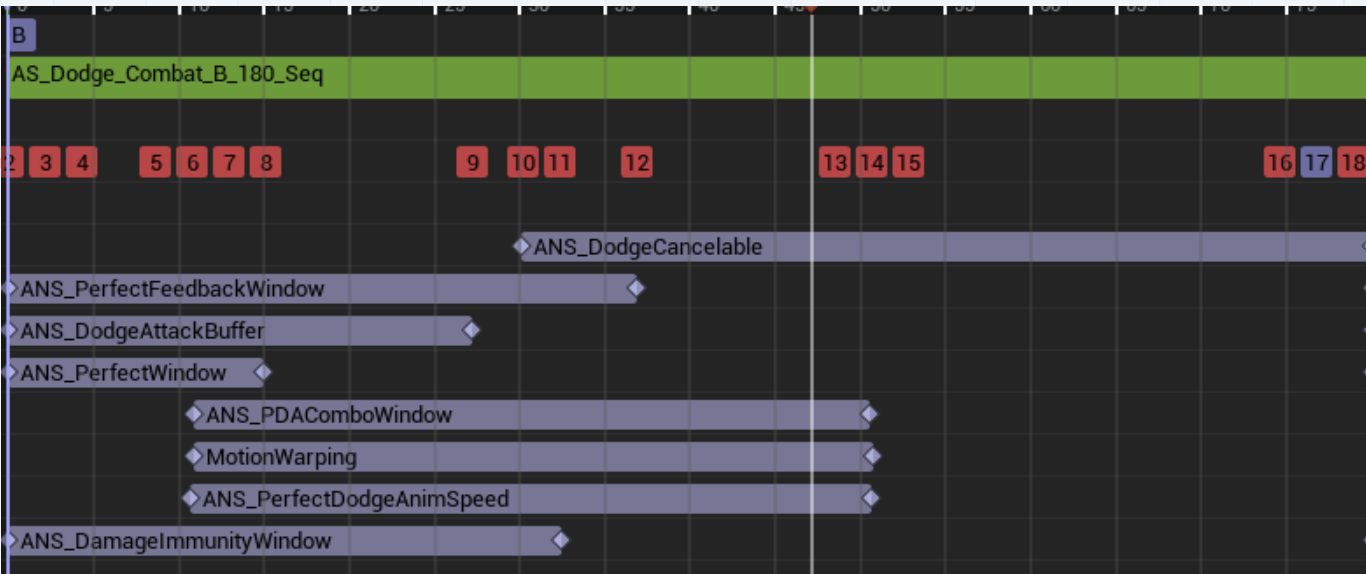
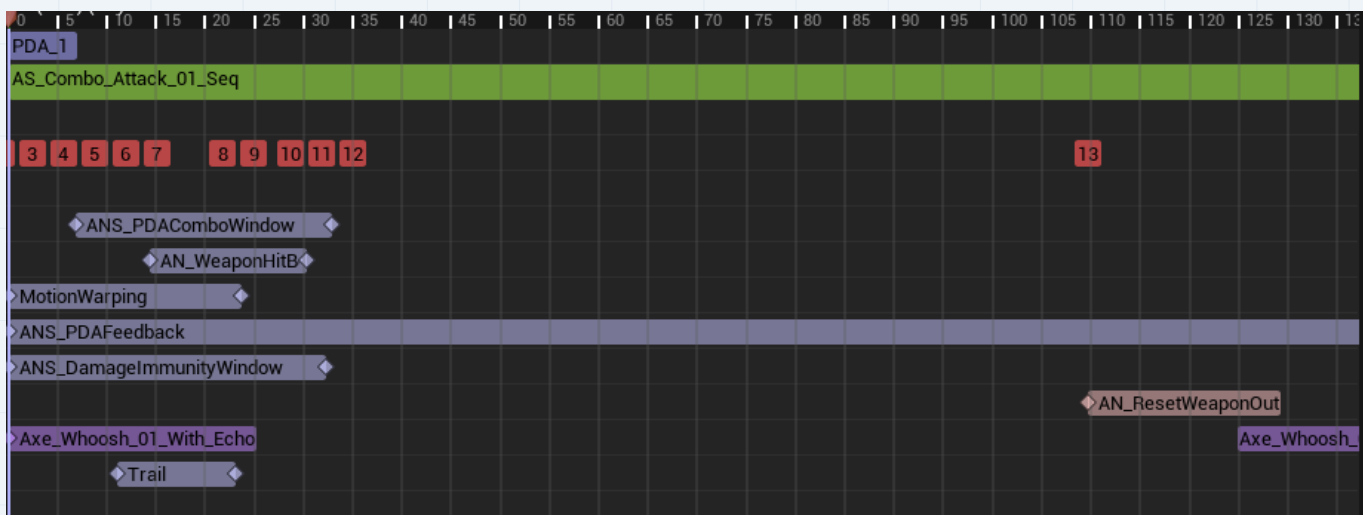


图 3C-10A Dodge / PerfectWindow 的 GA 入口与判定



Dodge 的动画状态通知



PerfectDodgeAttack 的动画状态通知

图 3C-10B 普通 Dodge 分支

CancelWindow 内的 GA 取消、重启与派生动作

2A. 未命中 PerfectWindow: 进入普通恢复段

ANS_CancelWindow

恢复帧打开后: UnlockBreathRecovery
bDodgeCancelWindowOpen = true
bMovementCancelWindowOpen = true

Attack -> DodgeAttack

TryTriggerDodgeAttack()
近: GA_DodgeAttack_Close
远: GA_DodgeAttack_Far
Tag: DodgeAttack.Close / Far

DodgeAttack: Close/Far GA

Space -> 再闪避

CancelDodge / DodgeFast
NextTick:
TryActivateAbilityByTag
GA_Dodge

Move -> 移动打断

StopAllMontages(0.15)
CancelAbilities:
Ability.Action.Dodge

Shift -> DodgeFast

TryActivate GA_Dodge_Fast
Tag: Ability.Action.DodgeFast
Cancel GA_Dodge
GE_Breath_Fast / Sprint

DodgeFast: Fast Tag + GE

图 3C-10B 普通 Dodge 恢复段: CancelWindow、GA 取消与分支

图 3C-10C Perfect Dodge / PDA

PDA Combo、Finisher、资源结算与清理

2B. 命中 PerfectWindow: 进入 Perfect Dodge / PDA

TriggerPerfectDodge()

清 State.PerfectWindow.Dodge
重置普通连段输入
播放完美闪避反馈

资源结算

FM = 1.0
BV -10 默认
AddYangForce
回血检查

开启 PDA 窗口

bPDAComboArmed = true
OpenPDAComboWindow
PressureWindow
Update WarpTarget

PDA 状态维护

PDA GA 是轻量入口
激活后调用 C++ 路由
随后 EndAbility
状态由 C++ Bool / Timer 维护

Light -> PDA Combo

GA_PDAAAttack 或输入路由
TryTriggerPDACombo()
CanBuffer -> NotifyEnd
PDA_Combo1~4 / PDA_1~4

Heavy -> PDA Finisher

GA_PDAFinisher 或 Heavy 路由
TryTriggerPDAFinisher()
ConsumeYangForce
AttackID: PDA_Finisher

PDA 结束: ResetPDACombo(), 清 Armed / Window / Buffer, 关闭反馈与 CameraShot

图 3C-10C Perfect Dodge / PDA: PDA Combo、Finisher 与清理

GAS 对应关系速查

时机	GA / 代码入口	Tag	GE / 属性与取消
按下 Dodge	GA_Dodge; 蓝图 Commit 后播放 Dodge Montage	Ability.Action.Dodge ; CoolDown.Ability.Do dge	GA_Dodge 资产引用 GE_Dodge_CoolDown 、GE_Breath_PerfectDodge_Normal、 GE_Breath_PerfectDodge_Peaked; 当前完美 闪避的 BV 改动由 C++ TriggerPerfectDodge 直接写入并记录 C++Direct。
PerfectWindow 打开	ANS_PerfectWindow(Do dge)	State.PerfectWindow. Dodge	只有 Ability.Action.Dodge 或 Ability.Action.DodgeFast 存在时才 AddLooseTag; NotifyEnd 或 TriggerPerfectDodge 会移除。

Perfect Dodge 成功	TriggerPerfectDodge()	清除 State.PerfectWindow. Dodge	FM 重置为 1.0; BV - PerfectDodgeBreathReduce; AddYangForce; TryApplyPerfectDodgeRecovery; 开启 bPDAComboArmed / bPDAComboWindowOpen。
普通 Dodge 恢复段	ANS_CancelWindow -> StartCancelCheck()	bDodgeCancelWindowOpen / bMovementCancelWindowOpen	解锁 BreathRecovery; Space 会取消 Dodge / DodgeFast 后重启 GA_Dodge; Shift 会激活 GA_Dodge_Fast; 移动会 StopAllMontages 并 Cancel Ability.Action.Dodge。
Dodge + Attack	TryTriggerDodgeAttack() -> GA_DodgeAttack_Close / Far	Ability.Action.DodgeAttack.Close 或 .Far	根据 SoftLockOnTarget 距离选择 Close/Far ; CurrentAttackData = EAttackID::DodgeAttack; GA 蓝图 Commit ; 失败时回退播放 AnimationData.FollowUp.DodgeAttack。
Dodge -> Fast	GA_Dodge_Fast	Ability.Action.DodgeFast	Shift + Space 或恢复段持续 Shift 触发; 取消 GA_Dodge; 资产引用 GE_Breath_Fast / GE_Breath_Sprint; 角色速度由运行态 bIsFasting / DodgeFast Tag 同步。
PDA Light	GA_PDAAAttack 或输入路由 -> TryTriggerPDACombo()	C++ 状态: bPDAComboArmed / bPDAComboWindowOpen / LateBuffer	不是持续 Tag 驱动; CanBuffer 时先缓存, NotifyEnd 执行 ExecuteBufferedPDACombo ; 读取 AnimationData.PDA.ComboSections 与 CombatData AttackTable。
PDA Combo 执行	ExecuteBufferedPDACombo()	AttackID: PDA_Combo1~4	设置 CurrentAttackData、CombatWarpTarget 与 Snap; PDA_Combo1/3/4 对目标施加 PDA 压力慢放 ; HandlePDAComboStarted 后关闭当前链, 等待下一次 Perfect Dodge 推进。
PDA Heavy / Finisher	GA_PDAFinisher 或 Heavy 路由 -> TryTriggerPDAFinisher()	AttackID: PDA_Finisher	IsPerfectFinisherYangReady 通过后 ConsumeYangForce; ResetPDACombo; 播放 AnimationData.PDA.Finisher; OnPDAFinisherMontageEnded 关闭反馈与 CameraShot phase。

3.5.3 状态路由表

前置状态	输入	达成状态	说明
常态	WASD	Move	普通移动。
移动中	按住 L_Shift	Sprint	进入加速移动。
Sprint	Space + 持续按住 L_Shift	Fast	Fast 必须由 Sprint+Dodge 并继续按住 Sprint 键触发。
Fast	LMB	Fast Attack	快移状态攻击。
常态/移动	Space	Dodge	普通闪避入口。
Dodge 窗口	LMB + 近距离	DodgeAttack_Close	根据距离选择近距离闪避追击。
Dodge 窗口	LMB + 远距离	DodgeAttack_Far	根据距离选择远距离闪避追击。
敌方攻击完美闪避窗口	Space	Perfect Dodge	进入 Perfect Dodge 状态。
Perfect Dodge 后	LMB	PDA Combo 01 + PreZen	触发第一段 PerfectDodgeAttack, 并开启 PreZen 倒计时。
PreZen 倒计时中	Perfect Dodge + LMB	PDA Combo 02	在倒计时内再次完美闪避并攻击, 推进第二段。
PDA Combo 02 后的 PreZen	Perfect Dodge + LMB	PDA Combo 03	同理推进第三段。
PDA Combo 03 后的 PreZen	Perfect Dodge + LMB	PDA Combo 04	同理推进第四段。
PDA Combo 01-04 任意招式	R	PerfectDodgeAttackFinisher	四段中的任意 PDA 招式都可按 R 进入终结。
完美格挡窗口	F	Perfect Parry	无需先右键防御, 直接按 F 触发。
Perfect Parry 窗口	LMB	ParryAttack	完美格挡后的反击攻击。

04 战斗设计

本章把战斗核心拆成四个互相驱动模块：呼吸机制、完美操作、阳之力、禅意时刻。它们共同服务于同一个体验目标：玩家在高压 Boss 战里不是追求无脑输出，而是在进攻欲望、身体惯性、精准反应和资源回收之间找到平衡。

战斗设计总览：四个互相咬合的系统

图标与数值来自当前代码：BreathAttributeSet / YangEnergyDataSet / ZenMomentDataSet / CharacterAnimationDataSet。



图 4-1 战斗设计四系统总览：呼吸、完美操作、阳之力与禅意时刻

4.1 呼吸系统

呼吸是「动作的惯性」，而非节拍器。贪刀和无效动作的积累会让气息从平稳走向紊乱，玩家需要在进攻欲望和身体惯性之间找到平衡。呼吸系统取代传统耐力条，是整场战斗的核心资源。

本节把呼吸系统拆成三组数学模型：呼吸值 BV 的行为变化、疲劳值 FM 的连续动作增长，以及 BV 恢复的条件代入。BV 决定当前压力区间，FM 决定重复动作的加权代价，恢复公式决定玩家脱离动作压力后能否回到安全区。

呼吸状态 (Breath State)

状态判定: $Zen = BV \leq ZenLimit$; $Agitated = ZenLimit < BV < PeakLimit$; $Peak = BV \geq PeakLimit$.			
默认参数: $ZenLimit = 80$, $PeakLimit = 99$; 代码 Tag 分别为 <code>State.Breath.Zen</code> / <code>State.Breath.Agitated</code> / <code>State.Breath.Peaked</code> .			
用途: 三种状态不是单纯的数值展示, 而是把 BV 压力翻译成玩家可感知的风险层级、反馈强度和系统触发条件。			
状态	BV 判定 / Tag	状态解释	设计用途
Zen	$BV \leq ZenLimit$ (默认 0~80) <code>State.Breath.Zen</code>	气息稳定, 动作压力仍在可控范围内; 允许自然恢复继续把 BV 往 0 拉。	作为安全输出和整理节奏的窗口; UI、镜头和受击反馈保持稳定, 鼓励玩家通过精准行动回稳。
Agitated	$ZenLimit < BV < PeakLimit$ (默认 81~98) <code>State.Breath.Agitated</code>	气息急促, 已经离开安全区; 自然恢复不再轻易生效, 后续增压会快速逼近 Peak。	作为风险预警层, 提醒玩家停止贪刀, 并通过 Perfect、Finisher 或防守窗口把压力压回 Zen。
Peak	$BV \geq PeakLimit$ (默认 99~100) <code>State.Breath.Peaked</code>	气息顶峰, 任何增压都容易封顶; 角色反馈进入高压表现, 受击与动作代价变得更明显。	作为惩罚与爆发边界; 限制无脑连接, 迫使玩家使用 PerfectDodge、PerfectParry 或 Finisher 解除高压。

呼吸值 (BreathValue / BV)

公式: $BV_{next} = clamp(BV_{now} + \Delta BV(行为), 0, 100)$ 。 增加型: $\Delta BV = BaseCost \times FM \times t$; 减少型: $\Delta BV = -ReduceAmount$ 。					
说明: BV_{now} 是行为发生前的呼吸值; BaseCost 是动作基础压力; FM 是疲劳倍率; t 是持续秒数; ReduceAmount 是完美操作/终结带来的回血量; clamp 保证 BV 始终在 0~100。					
状态列含义: Zen=安全区, Agitated=急促区, Peak=顶峰区; 单元格描述同一行为在该状态下会把 BV 推向哪里。					
行为	ΔBV 规则	Zen	Agitated	Peak	GAS / 代码入口
LightAttack	$+5 \times FM$	小幅上升; 临界时转 Agitated	继续增压; 推向 Peaked	封顶 100; 维持 Peaked	<code>Ability.Action.LightAttack</code> <code>ApplyBreathPressureToSelf</code> <code>GE_Breath_LightAttack</code>
HeavyAttack	$+12 \times FM$	明显上升; 更容易越界	快速接近 Peaked	封顶 100; 高风险	<code>Ability.Action.HeavyAttack</code> <code>ActionWeight=1.5</code> <code>GE_Breath_HeavyAttack</code>
DodgeAttack	$+4 \times FM$	轻度追击压力	缓慢推向 Peaked	可封顶; 需谨慎追击	<code>GA_DodgeAttack_Close/Far</code> <code>ApplyBreathCostToSelf</code>

Dodge	+10 固定	位移有固定压力	容易越过状态线	封顶 100; 维持 Peaked	Ability.Action.Dodge GE_Breath_Dodge / GE_Dodge_CoolDown
ChargeHold 1s	+8 × t	按秒持续上升	蓄力越久越接近 Peaked	持续蓄力会封顶	ChargeHoldBreathDrain 持续动作按秒累计
Fast / Sprint	+GE_Mag × t	持续状态增压	高速推进到 Peaked	维持封顶压力	GE_Breath_Fast GE_Breath_Sprint 持续状态驱动
PerfectDodge	-10	压回更稳; 不低于 0	可拉回 Zen	拉回 Agitated	TriggerPerfectDodge PerfectDodgeBreathReduce
PerfectParry	-15	压回更明显	更容易回到 Zen	拉回 Agitated	PerfectParryBreathReduce GE_Breath_PerfectParry_*
Finisher	-15	终结后释放压力	作为回稳出口	可脱离 Peaked	ApplyFinisherRecovery FinisherRecoveryBase
PreZen Perfect	-40	PreZen 内大幅压低 BV	PreZen 内推进 Zen 条件	PreZen 内快速脱离高压	ApplyPreZenPerfectAction PreZenPerfectDodge/Parry

呼吸值触发逻辑

行为输入 -> BV 变化 -> State.Breath Tag / UI / 恢复锁

触发流程

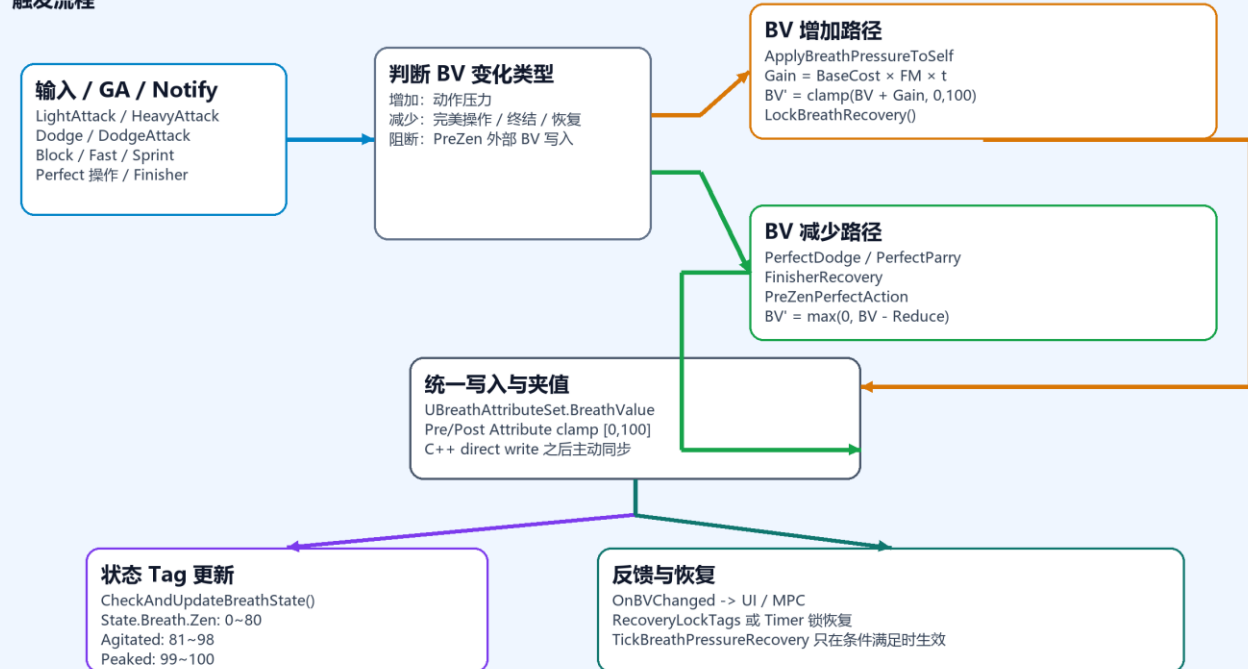


图 4-2A 呼吸值触发逻辑：行为输入、BV 写入、状态 Tag 与恢复锁

呼吸值恢复 (BreathValue Recovery)

公式：

- $FMA\alpha = \text{clamp}((FM-1)/(\text{MaxFM}-1), 0, 1)$;
- $\text{FatigueRecoveryScale} = \text{lerp}(1, \text{MinRecoveryMultiplier}, FMA\alpha)$;
- $\text{RecoveryRate} = \text{BaseRecoveryRate} \times \text{HealthModifier} \times \text{FatigueRecoveryScale}$;
- $BV_{\text{next}} = \max(0, BV_{\text{now}} - \text{RecoveryRate} \times \Delta t)$.

说明：

$FMA\alpha$ 把疲劳值归一化为 0~1; $\text{FatigueRecoveryScale}$ 根据疲劳程度在“完整恢复倍率 1”和“最低恢复倍率 $\text{MinRecoveryMultiplier}=0.15$ ”之间插值; RecoveryRate 是每秒实际恢复量; Δt 是经过时间; $\max$ 防止 BV 低于 0。

设计含义：FM 越高，恢复倍率越低，BV 下降越慢；完美操作和 PreZen 完美操作属于直接减少 BV，不走自然恢复 Rate。

特定情况	公式代入	设计含义
Peaked 完美操作	$BV=100$, PerfectDodge: $BV'=\max(0, 100-10)=90$; PerfectParry: $BV'=\max(0, 100-15)=85$ 。若触发 PeakedDodgeBoost: $FM'=\max(1, FM/3)$, 持续 0.4s。	从 State.Breath.Peaked 拉回 Agitated; 完美操作既降低 BV，也短暂减轻后续疲劳压力。
Zen 逐渐恢复	$BV=60$, $FM=1$, 满血 $\text{HealthMod}=1$: $\text{Rate}=5 \times 1 \times 1=5/s$, 1s 后 $BV'=55$ 。若 $FM=3.5$: $FMA\alpha=1$, $\text{Scale}=0.15$, $\text{Rate}=0.75/s$, 1s 后 $BV'=59.25$ 。	Zen 区间才允许自然下降; 疲劳越高，下降越慢，所以乱按后即使回到 Zen，也需要更长时间稳定。
PreZen BV 减少	$BV=70$, PreZenPerfectDodge/Parry Reduce=40: $BV'=\max(0, 70-40)=30$, 并增加	PreZen 中只有被标记的完美操作能主动压 BV; 普通动作压力

PreZenPerfectActionTimeBonus=1.25s。外部普通 GE 写 BV 会被 AttributeSet 冻结。

不会偷改 PreZen 的 BV 目标。

疲劳值 (FatigueMultiplier / FM)

公式: $\text{Count}[\text{action}] += \text{ActionWeight}$; $\text{Sigmoid} = 1 / (1 + e^{\{-0.8 \times (\text{Count} - 6)\}})$; $\text{FM} = \min(1 + 2.5 \times \text{Sigmoid}, \text{MaxFM}=3.5)$ 。

说明: Count[action] 是某类动作的连续计数; ActionWeight 是这次动作对疲劳的权重; Sigmoid 让疲劳前期缓慢、中段快速、后期接近上限; FM 是最终疲劳倍率, 会放大增加型 ΔBV。

状态列含义: Zen 里重复动作主要积累隐性风险; Agitated 中同样的 FM 会更快把 BV 推向 Peak; Peak 中 FM 让任何增压行为都更难脱险。

增加 BV 的行为	Zen	Agitated	Peak	GAS / 参数
LightAttack	1 次: FM 1.04, ΔBV 5.2 3 次: FM 1.21, ΔBV 6.0 6 次: FM 2.25, ΔBV 11.2	1 次: FM 1.04, ΔBV 5.2 3 次: FM 1.21, ΔBV 6.0 6 次: FM 2.25, ΔBV 11.2	6 次后 FM 2.25, 95BV 时一次轻击可顶到 100	Ability.Action.LightAttack Base=5, Weight=1.0
HeavyAttack	1 次: FM 1.07, ΔBV 12.8 3 次: FM 1.58, ΔBV 18.9 6 次: FM 3.29, ΔBV 39.5	1 次: FM 1.07, ΔBV 12.8 3 次: FM 1.58, ΔBV 18.9 6 次: FM 3.29, ΔBV 39.5	3 次后 FM 2.16, 95BV 时重击必封顶	Ability.Action.HeavyAttack Base=12, Weight=1.5
DodgeAttack	1 次: FM 1.04, ΔBV 4.2 3 次: FM 1.21, ΔBV 4.8 6 次: FM 2.25, ΔBV 9.0	1 次: FM 1.04, ΔBV 4.2 3 次: FM 1.21, ΔBV 4.8 6 次: FM 2.25, ΔBV 9.0	伤害追击低成本, 但重复仍推高 FM	GA_DodgeAttack_Close/Far Base=4
ChargeHold	1/3/6s: 按持续秒累计 BV +8/s; FM 若接入 Weight 则同步上升	高 BV 时蓄力会快速逼近 Peaked	持续越久越危险, 建议配明显反馈	ChargeHoldBreathDrain=8/s
Fast / Sprint	1/3/6s: GE 持续施压 曲线按等效 Weight=1.5/s 表现	Agitated 中 Fast 会更快推 到 Peaked	高 BV 使用 Fast 是高风险节奏	GE_Breath_Fast GE_Breath_Sprint
SprintAttack	按 GA_SprintAttack 的 BaseCost / Weight	与 Fast 状态叠加时更容易 放大 BV	高 BV 时作为终结入口而非循环手段	GA_SprintAttack ApplyBreathCost ToSelf

疲劳值曲线：Fast vs LightAttack

$$FM = \min(1 + 2.5 / (1 + e^{\{-0.8(\text{Count}-6)\}}), 3.5)$$

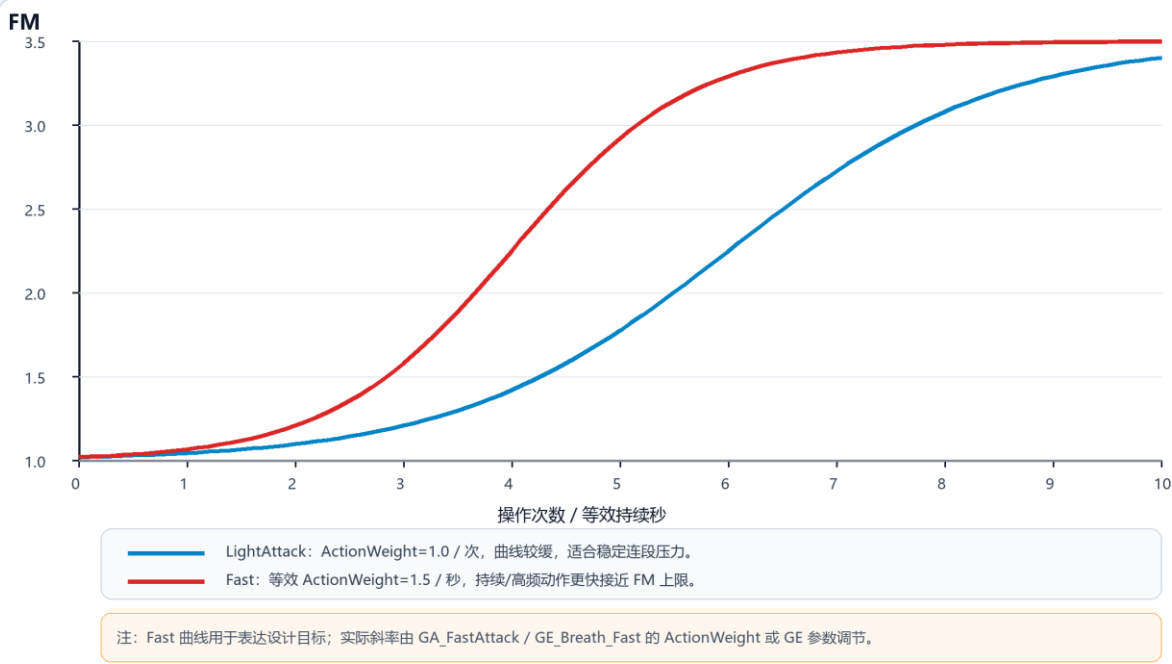


图 4-2B 疲劳值表现曲线：Fast 与 LightAttack 的 FM 增长

4.2 完美操作

完美操作是战斗节奏的调节阀，既是战斗攻防转换的枢纽也是让玩家从高危状态回到安全状态的机制

Perfect Parry 和 Perfect Dodge 二者都会清空疲劳惯性，并根据当前状态把玩家导向 ParryAttack、PDA 或 PreZen。

完美操作：防御、闪避、反击与 PDA 的路由

Perfect Action 是战斗节奏阀：成功后立刻清 FM，并根据当前状态进入反击或 PreZen 链路。

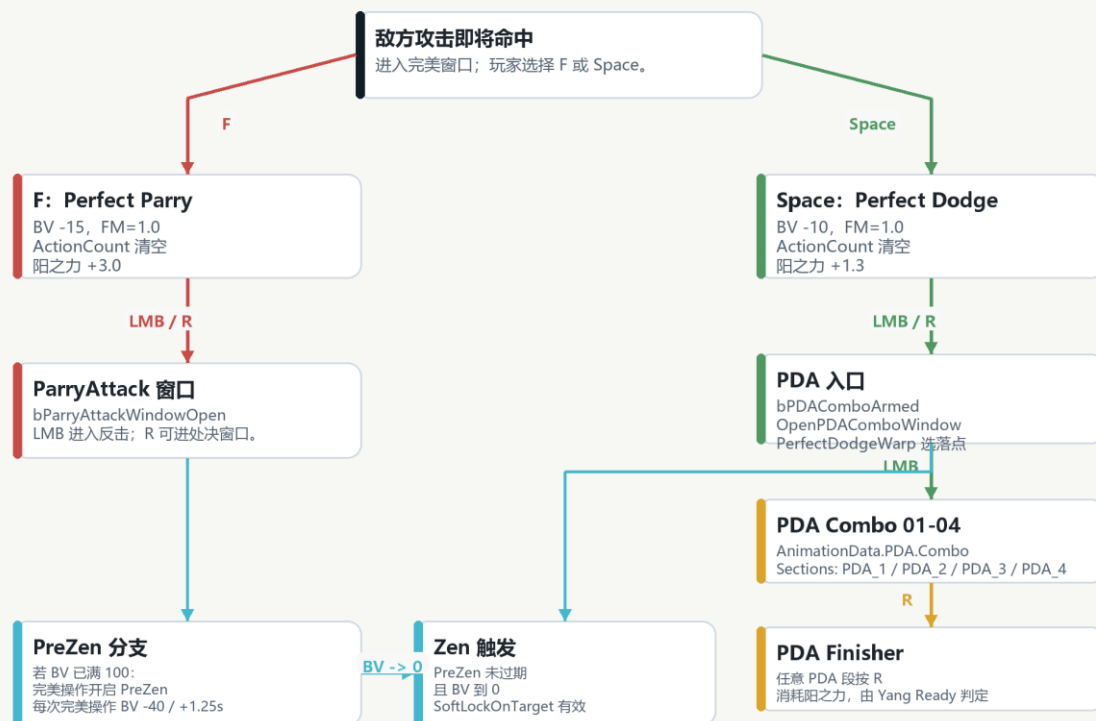


图 4-3 完美操作路由：Perfect Parry、Perfect Dodge、PDA 与 PreZen

入口	普通状态收益	后续动作	代码锚点
F / Perfect Parry	BV -15, FM=1.0, 阳之力 +3.0。	打开 ParryAttackWindow, 可接 LMB 反击或 R 处决。	HandleZenPerfectAction(false) TryTriggerParryAttack()
Space / Perfect Dodge	BV -10, FM=1.0, 阳之力 +1.3。	打开 DodgeAttackWindow、PDAArmed、Warp。	HandleZenPerfectAction(true) TryTriggerPDACombo()
PDA Combo	PDA_Combo1-4 由 AnimationData.PDA.Sections 配置。	每段窗口内 LMB 推进；任意段可按 R 进入 PDA_Finisher。	AnimationData.PDA EAttackID::PDA_Combo1-4
Perfect Action + BV 满	开启 PreZen, 后续完美操作每次 BV -40 并 +1.25s。	若 PreZen 未过期且 BV 到 0, 触发 Zen Moment。	BeginPreZenMoment() TryTriggerZenMoment()

4.2.1 完美操作的游戏循环

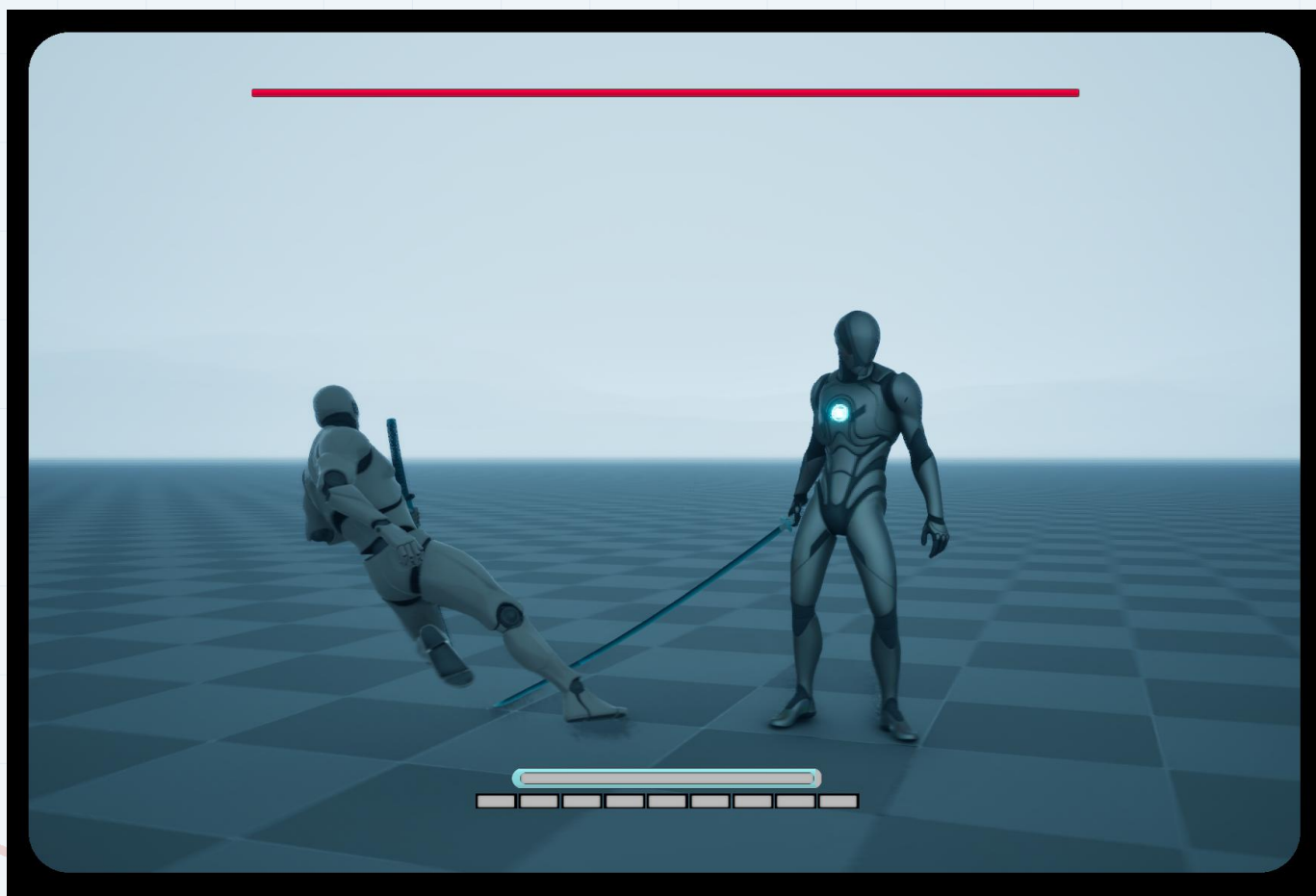
完美操作循环

Goal · Challenge · Reward

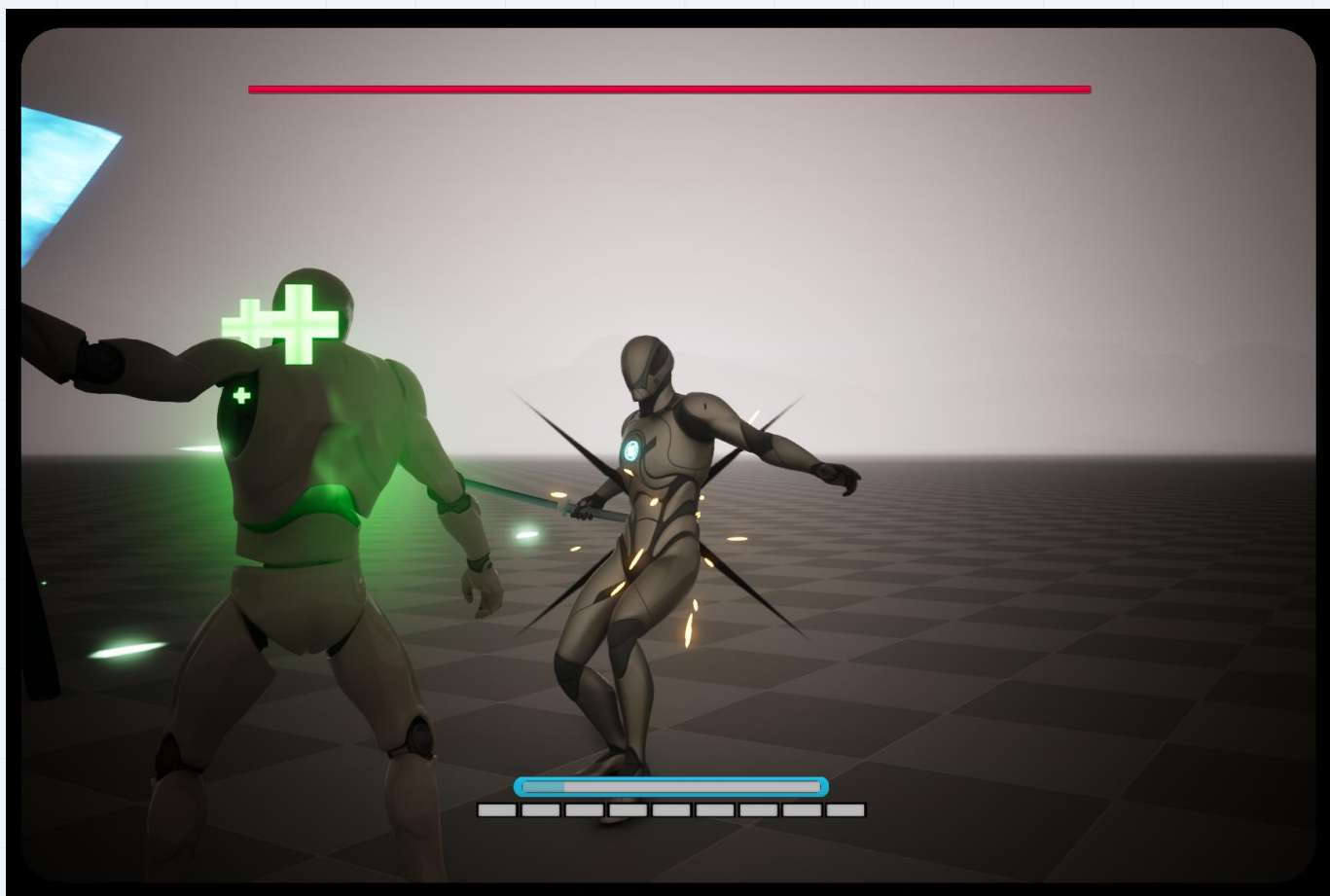
目标 触发完美操作

挑战 看准敌人攻击瞬间
做出对应按键

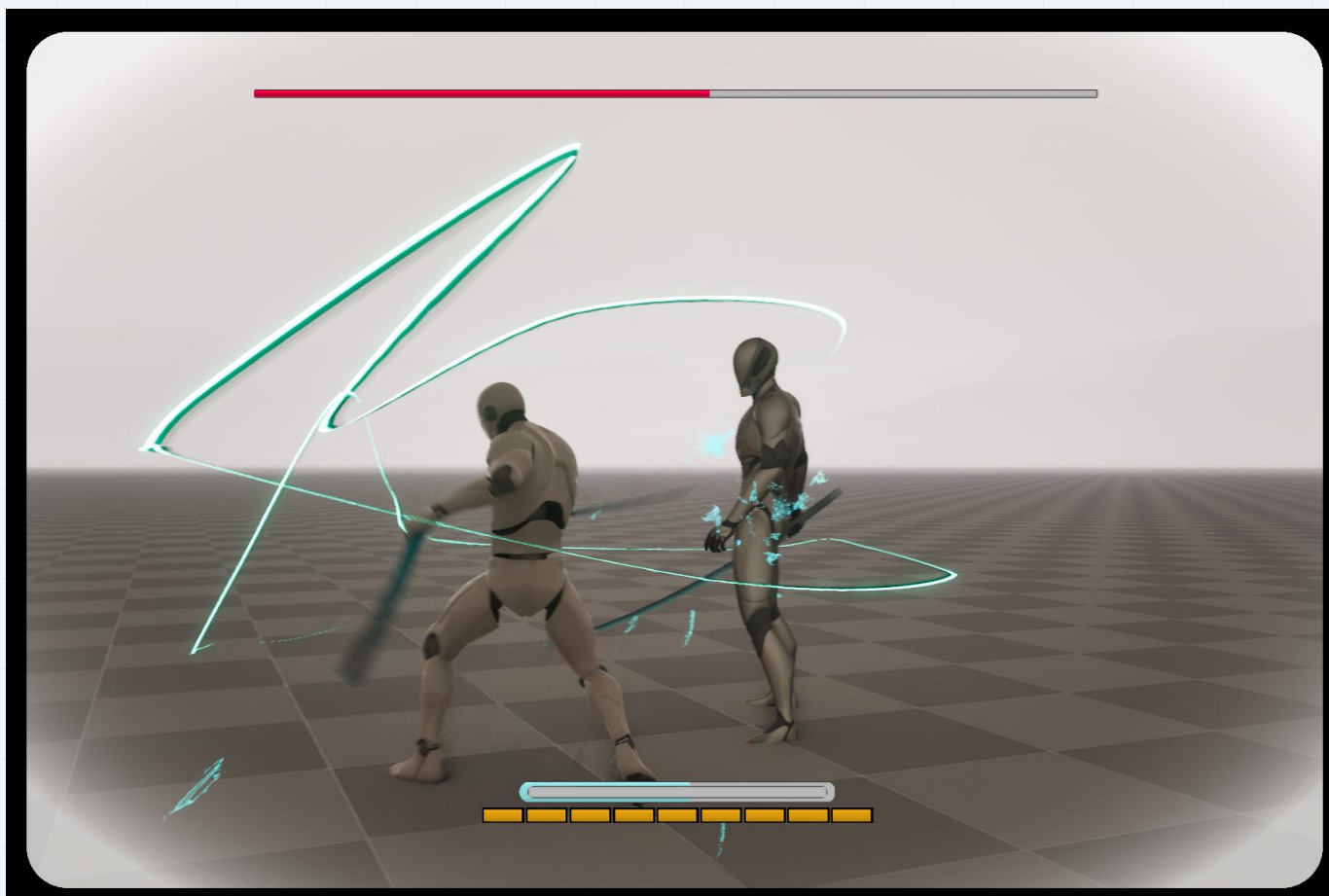
奖励 恢复血量，获得攻击窗口
夺回攻击主动权



完美闪避



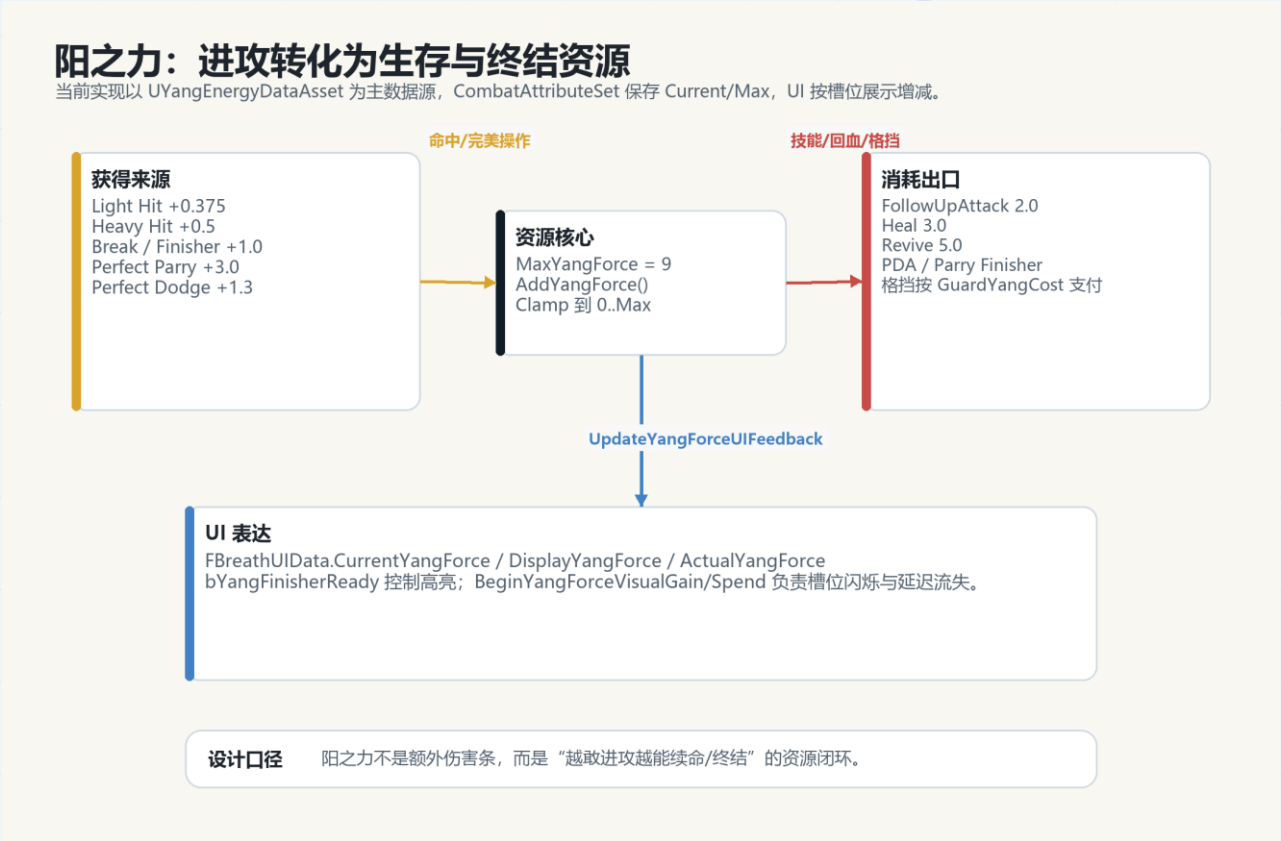
完美格挡



Prezen Attack

4.3 阳之力

阳之力表达的是「攻即生存」。玩家通过命中、破防和完美操作获得资源，再把资源花在回血、反击、终结和格挡承伤上，形成进攻到续命再到爆发的闭环。



CostHeal / Revive / FollowUp	3.0 / 5.0 / 2.0	回血、复活、后续攻击的主要支出。
YangForceToHealthRatio	0.05	格挡资源不足时，缺口可折算为生命损失。

4.4 禅意时刻

禅意时刻是完美操作链路的最高奖励，同时也是制作游戏心流和高级游戏循环的核心。

当前实现先进入 PreZen 倒计时窗口：玩家必须在窗口内继续用完美操作和命中维持节奏，并把 BV 压到 0；成功后进入 Zen Moment，目标深度慢放，玩家获得短暂的从容输出与恢复窗口。

禅意时刻：PreZen 倒计时到 Zen Moment

当前代码将寂静分成两个阶段：PreZen 是倒计时与降 BV 的资格窗口，Zen 是实际的慢放输出状态。

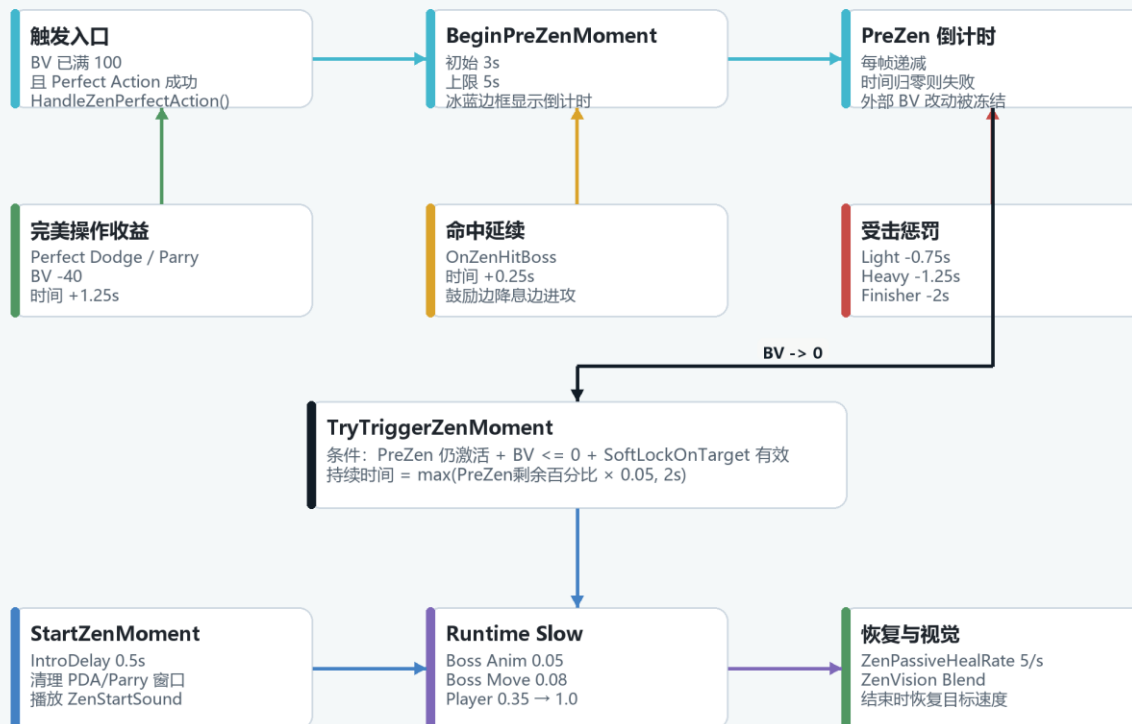


图 4-5 禅意时刻流程：PreZen 倒计时、BV 归零与 Zen Moment 慢放

阶段	参数/状态	设计作用
PreZen 开启	PreZenInitialDuration = 3s, Max = 5s。	给玩家一个短时间的“极限平息”窗口。
完美操作	PerfectDodge/Parry Reduce = 40 , TimeBonus = 1.25s。	真正推进 Zen 的主输入。

命中与受击	AttackHit +0.25s; Light/Heavy/Finisher 受击惩罚 0.75 / 1.25 / 2s。	鼓励主动但精准，惩罚失误。
Zen 持续	MinDuration = 2s; Ratio = 0.05 。	PreZen 剩余越多，Zen 回报越高。
Zen 慢放	TargetAnimRate = 0.05; TargetMovementRate = 0.08; PlayerRate 0.35 → 1.0。	敌人近乎停滞，玩家动作逐步恢复掌控感。
Zen 恢复	ZenPassiveHealRate = 5/s。	让“平静”拥有可感知的生存收益。

禅意时刻的游戏循环：



4.5 呼吸值，阳之力 UI 展示

呼吸值 UI 分成外侧的呼吸边框和内部的呼吸值

呼吸值 UI 的目标是让玩家在不读数字的情况下感知压力区间：外侧呼吸边框表达状态强度，内部进度条表达 BV 数值，PreZen Bar 只在极限窗口出现，受击反馈拥有最高显示优先级。

4.5.1 呼吸值 UI 展示图占位

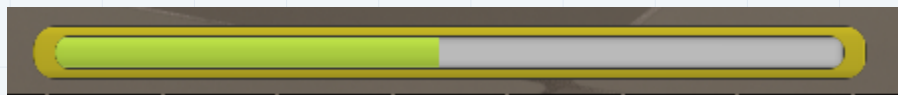


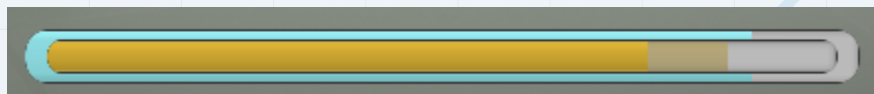
图 4-6A 呼吸值 UI 引擎截图占位：外侧呼吸边框、内部 BV 进度条与 PreZen Bar

4.5.2 呼吸值 UI 分层和优先级

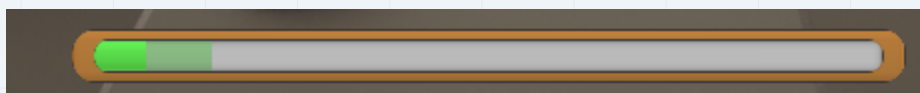
层级	展示内容	优先级规则
受击 / 失败反馈层	红闪、短震、裂纹、边框瞬时外扩；若处于 PreZen，同步显示扣时反馈。	最高优先级，短时间覆盖常态颜色和获取提示；反馈结束后回到当前 BV 状态。
PreZen Bar 层	Peak 后由完美操作触发的独立倒计时条，可贴近呼吸框外沿或置于 BV 条上方。	高于常态 BV 表现，低于受击反馈；只在 PreZen 激活期间显示。
呼吸边框层	外侧轮廓、呼吸光晕、Peak 脉冲速度；用于表达 Zen / Agitated / Peak。	持续显示，但不遮挡血量、锁定点、Boss 关键读招信息。
BV Progression Bar 层	内部填充条，展示 0-100 的当前 BV；支持渐变、拖影和回落动画。	常态主信息层；颜色与填充速度随 BV 区间变化。
底纹 / 刻度层	低透明背景槽、区间刻度、空槽边界。	最低优先级，保证其它反馈出现时仍能读出容量边界。

4.5.3 动态变化与 PreZen Bar 机制

进入 PREZEN 之后，外侧的边框会暂时变成冰蓝色，并作为一个计时器开始减少



受击状态，外侧边框会变成红色并闪烁一下



场景	UI 动态	机制说明
BV 上升	进度条向高压端推进；颜色从冷色稳定态逐步过渡到琥珀预警，再到 Peak 红色脉冲。	Zen $\leq$ 80, Agitated 81-98, Peak $\geq$ 99；颜色不是硬切，使用渐变让玩家读出接近危险线的过程。
BV 回落	主填充下降，保留一小段半透明拖影显示刚刚释放的压力；边框脉冲速度同步下降。	自然恢复使用已有 BV Recovery；完美操作和 Finisher 等直接减 BV 的行为可用更明显的回流动画。
PreZen Bar 激活	Peak 后完成完美操作时出现倒计时条；条体高亮但不替代 BV 条。	初始 3s，上限 5s；完美操作每次 BV -40 并 +1.25s。倒计时未空且 BV 到 0 时进入 Zen Moment。
PreZen 中命中 / 受击	命中时倒计时轻微回填并闪金；受击时条体被切掉一段并出现红色裂纹。	命中 +0.25s；轻 / 重 / 终结类受击扣时 0.75 / 1.25 / 2s，用 UI 明确告诉玩家窗口正在被惩罚。
受击表现	整体 UI 先短震，BV 条出现红色冲击波；如果处于 Peak，边框裂纹保留到下一次减压。	受击反馈优先级最高，但持续时间应短，避免遮挡血量变化、方向提示和角色受击动画。



呼吸值 UI 自上而下流程

阅读顺序固定为从上到下；右侧只补充覆盖规则，不中断主流程。



图 4-6B 呼吸值 UI 状态流：BV 写入、PreZen 倒计时、受击反馈与 Zen 入口

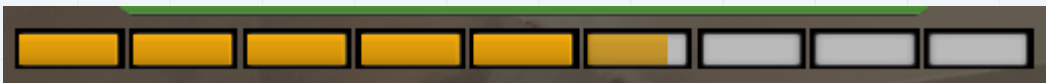
4.6 阳之力

阳之力 UI 的目标是把“攻即生存”的资源闭环即时化：玩家需要看见资源从命中进入槽位、从槽位预扣到能力、在格挡受击时被优先消耗，并在资源不足时明确转化为风险。

4.6.1 阳之力 UI 展示图占位



图 4-7A 阳之力 UI 引擎截图：9 槽资源、预览与不足警告



阳之力 UI 引擎截图占位：受击飘红与消耗预览

4.6.2 阳之力 UI 分层和优先级

层级	展示内容	优先级规则
受击 / 格挡消耗层	格挡命中时的槽位扣除、资源缺口、红色裂纹和 HP 警告。	最高优先级；让玩家明确知道这次防守消耗了多少资源，以及不足部分是否伤血。
主动使用层	Heal、Revive、FollowUp 等能力的消耗预览、确认扣槽和短暂锁定。	高于获取闪光；使用中先展示成本，再展示效果，避免玩家误读为资源增加。
获取反馈层	普通命中、破防、完美闪避、完美格挡带来的金色回填和浮动增量。	中优先级；可被受击或主动消耗覆盖，但覆盖后仍保留一次短脉冲提示。
槽位填充层	9 槽主资源，从左到右填充；小数收益可在下一槽内做半透明预填充。	常态主信息层；负责表达当前可用资源和距离下一槽的进度。

层级	展示内容	优先级规则
空槽 / 禁用层	空槽轮廓、能力不可用锁定、低透明背景。	最低优先级；只提供容量边界，不抢主反馈

4.6.3 获取、使用与受击动态

场景	UI 动态	机制说明
获取阳之力	资源从命中方向或角色中心回流到槽位；新增加的部分用金色闪光标记，随后收敛到常态填充。	普通轻击 +0.375，重击 +0.5，破防 / 终结类命中 +1.0；完美格挡 +3.0，完美闪避 +1.3。
预使用	按下或进入可用窗口时先显示成本投影：将要消耗的槽位变为高亮描边。	Heal 3，Revive 5，FollowUp 2；资源不足时对应能力图标和槽位边缘变红，不播放消耗动画。
使用阳之力	确认后槽位从右向左熄灭，被消耗的槽位产生金色流线进入角色或武器。	使用反馈高于获取反馈；短暂锁定被扣槽位，等能力效果开始后再回到普通可读状态。
格挡受击	若阳之力足够，槽位按消耗量切除并叠加盾形冲击；若不足，缺口段变红并连到血量警告。	格挡资源不足时，MissingYang 可按 YangForceToHealthRatio 折算为 HP 伤害，UI 必须让玩家看懂伤血来自资源缺口。
普通受击	阳之力槽整体短震、降亮 0.2-0.35s；不播放扣槽，避免误导。	非格挡受击不直接消耗阳之力；受击反馈应让位于血量、方向和 HitReact 表现。

阳之力 UI 自上而下流程

三条路径并行阅读：获取、主动使用、格挡受击；所有路径最终回到 9 槽常态显示。

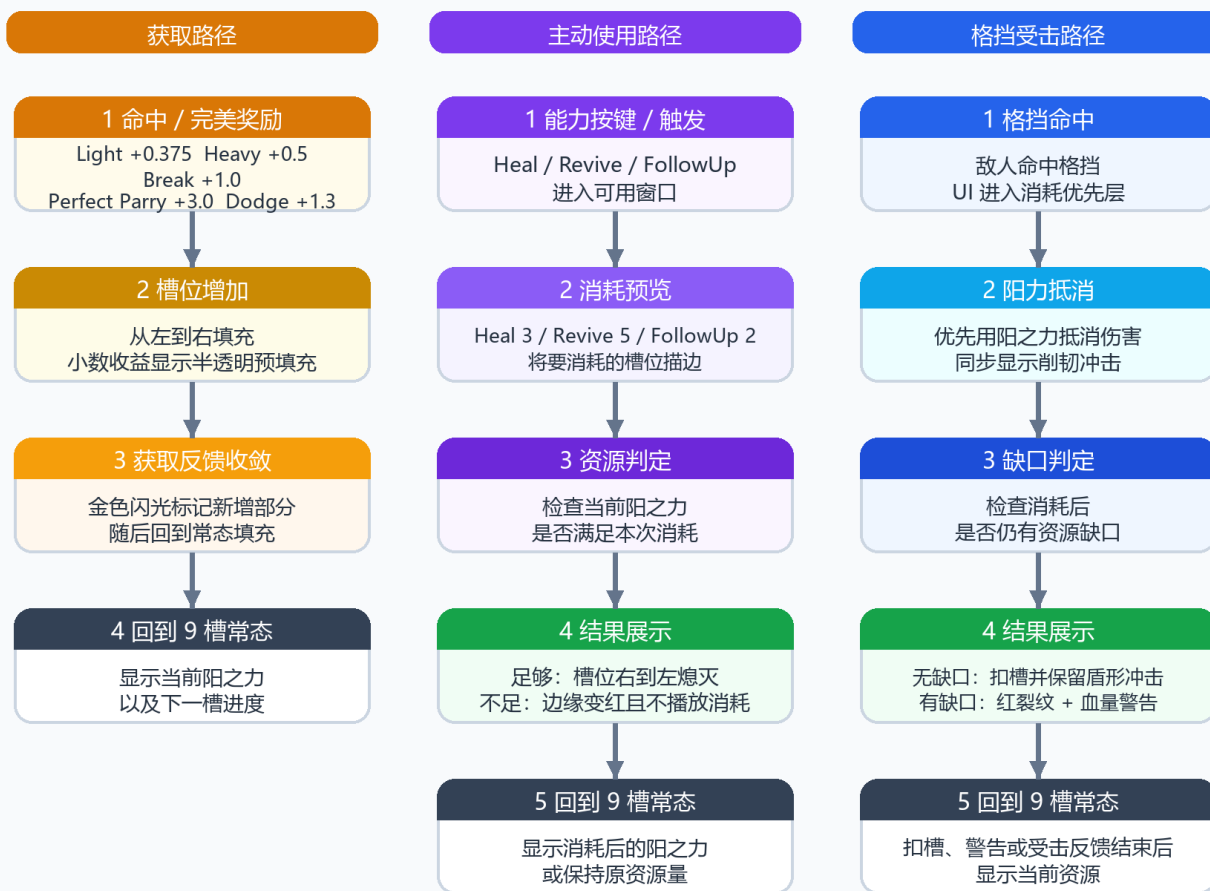


图 4-7B 阳之力 UI 状态流：获取、预消耗、使用确认、格挡受击与资源不足

05 伤害链条与伤害反馈

本章把“命中”拆成两条并行链：一条是数值链，负责从攻击数据计算出 HP、韧性、BV 与破防结果；另一条是反馈链，负责把同一次命中翻译成 HitReact、镜头、顿帧、UI 刀痕、血量边缘、VFX/SFX 与攻击者收益。反馈选择以 AttackDefinitionDataAsset / FAttackData 中的 AttackID、AttackLevel 与命中参数为源头，再进入具体表现库。设计目标是让每次受击都有明确的判定顺序、可追踪的代码入口，以及可调的 GAS/数据资产落点。

5.1 伤害判定与反馈开始的流程图

流程从攻击 Montage 的 HitBox/Notify 进入 `TakeHit`，先处理 PerfectDodge、PerfectParry 与免伤窗；如果目标正在格挡，则进入 Guard 分支；否则进入普通伤害分支。数值写入完成后，反馈链才开始播放，避免表现先于结算。

伤害判定与反馈开始流程

HitBox / Notify -> Perfect 与防御判定 -> 伤害结算 -> HitReact / UI / VFX / Camera

1. 命中入口

Weapon HitBox / Notify

攻击 Montage 的命中帧开启 HitBox
OnWeaponHitBoxOverlap 收集 Target
传入 AttackData / HitLocation
FXLocation / SwingDirection

Target.TakeHit(...)

接收攻击者、攻击数据与命中位置
读取 ASC Tag: Dodge / Parry / Block
读取状态: 死亡、免伤窗、Peak、PreZen

2. 前置否决

Perfect / 免伤?

DodgeWindow / ParryWindow
DamageImmunity

YES

NO

否决伤害

TriggerPerfectDodge()
TriggerPerfectParry(Attacker)
或 IsDamageImmunityWindow
Active()

Block 分支?

Ability.Action.Block
Guard Yang Cost

YES

NO

Guard 结算

CalculateHitResult 使用 Block
DmgMul / BlockPoiseMul
消耗 YangForce; 不足转 HP
GuardBreak 时 EnterHitReact

3. 计算与写入

CalculateHitResult

读取 CombatData.GetHitConfig
FinalDamage / FinalPoiseDamage
HealthImpact / PoiseImpact / ImpactScore
解析 HitReactIntent

属性写入

UpdateBVOnHit(BVDelta)
CurrentToughness -= FinalPoiseDamage
CurrentHealth -= FinalDamage
死亡 / 破韧 / HeavyHit 判定

4. 反馈开始

受击可视反馈

AttackData.AttackID / Level
驱动命中反馈配置
DamageIndicator / HealthFeedback
NearDeath RedFlash / Blood Flecks

HitReact 控制

PlayHitReactMontage
EnterHitReact 添加 Status.HitReact
取消攻击 GA; 阻断 Block GA
Montage BlendOut -> ExitHitReact

攻击者命中反馈

OnHitLanded.Broadcast(HitResult)
命中帧 / 镜头冲击 / FOV
YangForce Gain / PDA LifeSteal
HitStop 参数来自 FAttackHitConfig

GAS 对应: 当前扣血/削韧由 C++ 直接写 AttributeSet; 反馈选择以 AttackData.AttackID / AttackLevel 为索引。后续 GA 命中帧可创建 GE_Damage_Instant / GE_ToughnessDamage, 并通过 SetByCaller 写入。

图 5-1 伤害判定与反馈开始流程：前置否决、格挡、伤害写入与反馈启动

5.2 伤害流程的代码框架

代码框架按“攻击者命中帧 -> 受击者判定 -> 数值结算 -> 反馈启动 -> 攻击者命中回调”组织。当前工程主要由 C++ 直接写属性，GAS 侧已具备 Tag 与 SetByCaller 的接入点，后续可把扣血、削韧从直接写属性迁移成 GE。

```
// 入口：攻击帧开启 HitBox 后，Overlap 把目标、AttackData 和命中信息传给受击者。
OnWeaponHitBoxOverlap(Target)
{
    Target->TakeHit(this, CurrentAttackData, bTargetIsPlayer, HitLocation, FXLocation,
SwingDirection);
}

TakeHit(Attacker, AttackData, bIsPlayerReceiving, HitLocation, FXLocation,
SwingDirection)
{
    if (bIsDead) return;
    Read ASC Tags: Ability.Action.Dodge / Parry / Block / State.PerfectWindow.*;

    if (Dodging && State.PerfectWindow.Dodge) { TriggerPerfectDodge(); return; }
    if (Parrying && State.PerfectWindow.Parry) { TriggerPerfectParry(Attacker); return; }
    if (IsDamageImmunityWindowActive()) return;

    OnPreTakeHit(...);           // Boss 防御 AI 可在这里补 Block Tag

    if (HasTag(Ability.Action.Block))
    {
        GuardResult = CalculateHitResult(AttackData, bIsPlayerReceiving);
        PlayReceiveImpactFeedback(GuardResult, HitLocation, true);
        EmitGuardBlockDamageIndicator(...);
    }
}
```



```

    if (GuardBreak) EnterHitReact(GuardBreakMontage, BreakReact);
    Attacker->OnHitLanded.Broadcast(GuardResult);
    return;
}

```

```

HitResult = CalculateHitResult(AttackData, blsPlayerReceiving);
ApplyPeakHitReactRules(HitResult, ...);
BuildPlayerDamageIndicatorIfNeeded(HitResult);
ApplyHitResult(HitResult, Attacker, AttackerLocation, HitLocation);
Attacker->OnHitLanded.Broadcast(HitResult);
}

```

代码链条原则：TakeHit 只决定路线；CalculateHitResult 产出和写入数值；ApplyHitResult 只启动表现和状态；OnHitLanded 把命中收益返还给攻击者。

阶段	当前代码入口	职责	GAS / Tag / GE 对应	输出
攻击命中帧	EnableWeaponHitBox() OnWeaponHitBoxOverlap()	攻击 Montage 在有效帧 打开 HitBox, 收集 Target、HitLocation、 FXLocation、 SwingDirection。	GA_LightAttack / GA_HeavyAttack / GA_DodgeAttack Ability.Phase.Settlement SetByCaller.Damage (预留)	Target.TakeHit(...)
受击入口	AYinYangCharacterBase::TakeHit(...)	统一处理死亡、Perfect、 Parry、Dodge、Block、 免伤窗和 Peak 前置状态。	Ability.Action.Dodge Ability.Action.Parry Ability.Action.Block State.PerfectWindow.Dodge / Parry	进入否决、格挡 或普通伤害分支
数值计算	CalculateHitResult()	读取 CombatData.GetHitConfig, 计算 FinalDamage、FinalPoiseDamage、	当前: C++ 直接写 AttributeSet 目标: GE_Damage_Instant / GE_ToughnessDamage	FHitResultData

		BVDelta、ImpactScore 与 HitReactIntent。	SetByCaller.Damage / SetByCaller.PoiseDamage	
格挡结算	TakeHit() Block branch EmitGuardBlockDamageIndicator()	按 BlockDamageRatio、BlockPoiseDamageRatio 与 GuardYangCost 结算；不足阳之力转 HP 伤害。	Ability.Action.Block Status.HitReact (GuardBreak 时) GE_GuardCost (建议)	GuardHit / GuardBreak / Guard VFX
结果应用	ApplyHitResult()	播放受击表现，处理 Death、Break、Camera、BV Flash、PreZen 受击惩罚。	Status.HitReact Cancel Ability.Action.LightAttack / HeavyAttack Block Ability.Action.Block	HitReact Montage / Camera / UI
攻击者回调	OnHitLanded.Broadcast(HitResult)	让攻击者得到命中顿帧、镜头反馈、阳之力获取、PDA 吸血等收益。	GameplayEvent.HitLanded (建议) SetByCaller.YangGain	HitStop / YangForce / PDA LifeSteal
反馈配置	AttackDefinitionDataAsset FAttackData AttackID / AttackLevel	AttackData 决定这次命中属于哪一招、哪一级反馈；HitResult 再把 HealthRatioAfter、HitDirection、FinalDamage 传给表现层。	当前：按 AttackID 解析命中反馈库 建议： GameplayCue.HitImpact / DamageIndicator	世界特效、屏幕刀痕、血边、濒死红闪

5.3 伤害计算公式

公式总览：伤害链条先计算 FinalDamage / FinalPoiseDamage，再写入 CurrentHealth / CurrentToughness，最后用 ImpactScore 和 HealthRatioAfter 决定反馈级别。

计算项	公式	含义	代码 / 数据来源
原始伤害	RawDamage = AttackData.Damage ×	攻击数据先乘攻击等级配置，再扣除目标防御。	FAttackData.Damage FAttackHitConfig.Dama

	$\text{DamageMultiplier} - \text{Defense} \times \text{DefenseMultiplier}$	DefenseMultiplier 控制防御对该攻击等级的有效性。	geMultiplier UCombatAttributeSet. Defense
最终 HP 伤害	$\text{FinalDamage} = \max(\text{MinDamage}, \text{RawDamage} \times \text{BlockDmgMul})$	非格挡时 BlockDmgMul=1; 格挡时使用 CombatData.BlockDamageRatio。没有攻击数据或 Damage<=0 时 FinalDamage=0。	CalculateHitResult() FAttackHitConfig.MinDamage CombatData.BlockDamageRatio
格挡阳气抵扣	$\text{MissingYang} = \max(0, \text{GuardYangCost} - \text{ConsumedYang})$ $\text{GuardDamage} = \text{MissingYang} \times \text{YangForceToHealthRatio} \times \text{MaxHealth}$	玩家格挡先消耗阳之力; 阳之力不足时, 不足比例转为 HP 伤害。	GuardYangCost ConsumeYangForce() YangEnergyData.YangForceToHealthRatio
削韧伤害	$\text{BasePoise} = \text{AttackData.PoiseDamage} > 0 ? \text{AttackData.PoiseDamage} : \text{HitConfig.PoiseDamage}$ $\text{FinalPoiseDamage} = \text{BasePoise} \times \text{PoiseMultiplier} \times \text{BlockPoiseMul}$	韧性伤害优先用招式数据; 没填时用攻击等级配置。格挡时按 BlockPoiseDamageRatio 降低。	FAttackData.PoiseDamage FAttackHitConfig.PoiseDamage CombatData.BlockPoiseDamageRatio
属性写入	$\text{CurrentHealth}' = \max(0, \text{CurrentHealth} - \text{FinalDamage})$ $\text{CurrentToughness}' = \max(0, \text{CurrentToughness} - \text{FinalPoiseDamage})$	数值链的最终落点。当前由 C++ 直接 Set; GAS 化后应由 GE 修改并由 AttributeSet clamp。	UCombatAttributeSet. CurrentHealth UCombatAttributeSet. CurrentToughness PostGameplayEffectExecute clamp
影响评分	$\text{HealthImpact} = \text{clamp}(\text{FinalDamage} / \text{MaxHealth}, 0, 1)$ $\text{PoiseImpact} = \text{clamp}(\text{FinalPoiseDamage} / \text{MaxToughness}, 0, 1)$	ImpactScore 决定轻受击/重受击倾向; 攻击等级偏置让重击、终结技更容易触发强反馈。	FHitResultData.HealthImpact PoiseImpact ImpactScore

	ImpactScore = max(HealthImpact, PoiseImpact) + AttackLevelBias		
重受击窗口	WindowSum = Σ RecentPoiseDamage within HeavyHitWindow HeavyHit = WindowSum >= MaxToughness × HeavyHitThreshold 或 ImpactScore >= HeavyHitImpactThreshold	连续削韧会在滑动窗口内累计， 避免多段轻击完全没有打断感。	RecentPoiseDamage CombatData.HeavyHit Window HeavyHitThreshold
反馈强度	HealthRatioAfter = CurrentHealth' / MaxHealth SizeScale/Duration = Tuning[AttackLevel] NearDeath = HealthRatioAfter <= NearDeathThreshold	血量比例与攻击等级共同决定 UI 刀痕大小、持续时间、血边、濒 死红闪和低血量镜头。	FPlayerDamageIndicat orData UHealthFeedbackData Asset.DamageIndicator NearDeath

带入示例用于说明公式语义，实际数值以 CombatData、AttackDefinitionData 与角色属性为准。

情况	带入示例	结果解释
普通轻击	Damage=12, DamageMultiplier=1, Defense=5, DefenseMultiplier=1, MinDamage=1 FinalDamage=max(1,12-5)=7	低伤害但仍会刷新血量反馈；若削韧不高， 通常进入 LightHit。
格挡但阻力足 够	GuardYangCost=0.01875, CurrentYang>=Cost MissingYang=0, FinalDamage=0 FinalPoiseDamage 按 BlockPoiseDamageRatio 结算	不扣血，但仍有 GuardHit、格挡火花、镜 头轻震和削韧。
格挡且阻力不 足	CurrentYang=0.01, Cost=0.01875 MissingYang=0.00875	把资源缺口转成 HP 伤害，避免无资源格挡 仍然完全安全。

	GuardDamage=MissingYang × YangForceToHealthRatio × MaxHealth	
破韧 / Stun	CurrentToughness - FinalPoiseDamage ≤ 0	设置 bToughnessBreak 与 bTriggerBreak，非格挡进入 Stun，格挡进入 GuardBreak，并启动 ToughnessRecovery。
重受击	WindowSum ≥ MaxToughness × HeavyHitThreshold 或 ImpactScore ≥ HeavyHitImpactThreshold	即使单次伤害不致命，多段连续削韧也能触发 HeavyHit 表现。
死亡	CurrentHealth' ≤ 0	HitReactIntent=Death，取消 Ability，停止移动与碰撞；Boss 可进入处决窗口。

5.4 受击反馈类型与展示

反馈类型	触发条件	展示内容	Tag / 状态
PerfectDodge / PerfectParry	处于 Dodge/Parry 且命中帧落在 State.PerfectWindow.* 内	本次伤害被否决；播放完美反馈、镜头冲击、慢动作或反击窗口。	State.PerfectWindow.Dodge State.PerfectWindow.Parry
DamageImmunity	ANS_DamageImmunityWindow 增加免伤计数期间	无扣血、无削韧、无 HitReact；用于处决、起身或特殊动作保护。	DamageImmunityWindowCount > 0
GuardHit	目标拥有 Ability.Action.Block 且未破防	格挡火花 / Guard VFX，轻镜头，GuardHit Montage；玩家侧显示格挡命中提示。	Ability.Action.Block
GuardBreak	格挡时 CurrentToughness ≤ 0	GuardBreak Montage、BreakReact 状态、强镜头，格挡能力被打断。	Status.HitReact EBossCombatState.BreakReact

LightHit	ImpactScore 低于重受击阈值，且未被强制 Heavy/Knockback	按方向播放 LightHit/Additive Montage，小刀痕、小血边、短顿帧。	EHitReactIntent.LightHit
HeavyHit	ImpactScore 达阈值，或 HeavyHitWindow 内累计削韧达阈值	重受击 Montage，较大刀痕、强镜头、长顿帧；可明显打断节奏。	EHitReactIntent.HeavyHit
Stun / ToughnessBreak	非格挡 CurrentToughness <= 0 或 bTriggerBreak=true	Stun Montage；玩家可禁用输入，Boss 进入 Stunned；恢复由 ToughnessRecoveryTimer 拉回。	EHitReactIntent.Stun EBossCombatState.Stunned
Knockback	AttackData.Forced HitReact = Knockback 或特殊招式配置	击退起跳、空中循环、落地/起身 Montage；位移方向来自 KnockbackDirection。	EHitReactIntent.Knockback
Death / Execution	CurrentHealth' <= 0	死亡 Montage 或 Boss 被处决窗口；取消 Ability、停移动、关碰撞。	EHitReactIntent.Death EBossCombatState.Dead
血量与濒死反馈	玩家受伤后 HealthRatioAfter 更新；低于 NearDeathThreshold	屏幕刀痕、血边、Blood Flecks、灰度、濒死红闪、循环低血量镜头。	NearDeathFeedbackActive
命中点 VFX / SFX	普通命中或格挡命中均会触发	由 AttackData.AttackID / AttackLevel 决定反馈索引；可贴骨骼或命中点生成，声音可随机选取。	GameplayCue.HitImpact（建议）



普通受击：少量血浆+伤害边框



重伤状态：中量血浆+伤害边框+暗角+少量黑白滤镜



濒死状态：大量血浆+伤害边框+暗角+黑白滤镜

06 AI 设计

6.1 敌人设计思路

Boss 的设计灵感来自高强度一对一动作战斗中的“对峙感”：

玩家需要读距离、读前摇、管理呼吸值，并在进攻、防御、闪避和完美防御之间切换。Boss 不应该像随机播放招式列表的机关，也不应该像完全预测玩家行为的黑箱，而应像一个带有明确战斗规则的对手。

具体体现在：

- 近战压迫来自站位和招式范围，而不是纯粹提高攻击频率。
- 反击和防御用于阻止玩家无成本贴身连打，但必须有清晰前摇、硬直或状态反馈。
- 阶段推进让后半段更紧张：低血量阶段减少受击后的安全窗口，提高 Boss 回到主动权的速度。
- AI 的出招不能是随机，也不能重复。
- 整个战斗过程要有曲折，要让玩家感受到战斗从平淡到高潮的转变
- AI 的“聪明”要落在可读的树分支和参数上，方便调试、复盘和后续平衡。

因为我们的战斗设计是让玩家保持“低呼吸感”，所以在战斗设计中的博弈和对峙感会充分围绕玩家“呼吸值”来进行设计。敌人的目的是通过高伤害和高频率的攻击给玩家压力，而玩家的战斗则是想尽办法让自己呼吸值保持健康水平。

6.2 设计目标

目标	设计含义	工程落点
BT 可控	普通攻击由 BT 分支明确选择，不把主决策藏在不可见的自动评分里。	BTask_SetAttackIntentCpp 写入 AttackIntent; BTask_ExecuteAttackCpp 默认要求 AttackID 或 AttackIntent。
状态先行	死亡、受击、硬直、格挡、反击等高优先级状态要先于普通进攻。	BossCombatState、PostHitReactRecovery、Defend/ParryDefend、CounterStrike 作为上层门控。
距离绑定	每个招式只在合适距离执行，避免远距离原地挥空或近距离错误冲刺。	AttackData 配置 Close/Mid/Far 允许范围；MoveToPlayerAttackRange 和 ExecuteAttack 双重校验。
GAS 统一执行	BT 不直接播放蒙太奇，只负责调度。真正招式数据、蒙太奇变体和 Motion Warping 由能力层处理。	Controller 写 PendingAttackID; GA_BossAttackBase 从 CombatData 查询 FAttackData 并执行。
压力可调	CP 信号作为调参和可选分支依据，不能把未使用的历史统计写成当前核心策略。	BTS_UpdateCP 同步 DistanceRange、Greedy/Turtle、bShouldCounterStrike 等；普通选招仍以 BT 意图为准。

6.3 战斗难点

AI 的难点不在于让 Boss 永远更快，而在于让玩家清楚知道自己为什么被压制。当前结构的挑战主要来自距离、状态优先级、冷却、反击和阶段恢复时间的叠加。只要其中一个环节过强，玩家就会觉得 Boss 不讲理；只要其中一个环节过弱，玩家又可以用单一套路压住 Boss。

难点	玩家感受	AI 如何制造
距离压力	玩家要判断贴身、后撤、侧移或准备完美防御。	不同 BT 分支绑定 Close/Mid/Far, MoveToPlayerAttackRange 保证出招前到位。
反击压力	连续输出会让 Boss 更可能回击，不能只靠贴身连打取胜。	PatientDamageToFill、bShouldCounterStrike 和 CounterStrike 形成被压制后的回击机制。
防御循环	Boss 不会一直挨打，玩家需要识别格挡和招架窗口。	Defend/ParryDefend 用 Block Tag 和蒙太奇表现防御状态。
受击后恢复	后半段 Boss 受击后的安全窗口变短。	Normal/Hard/Power 阶段可用不同 PostHitReactRecovery 时长。
配置一致性	同一招式的移动、范围检查和 GA 表现必须一致。	AttackIntent 让 MoveTo、ExecuteAttack 和 CombatData 共享同一 AttackID。

6.4 难度平衡

难度平衡的原则是先调结构入口，再调数值强度。当前工程最值得优先调的不是隐藏评分，而是 BT 分支出现条件、AttackData 距离范围、攻击冷却、阶段恢复时间、Patient 阈值和防御持续时间。这样每次调整都能对应到玩家实际感受到的压力来源。

调参手柄	当前口径 / 建议	影响
AttackCooldown	默认约 2.5s, 可按分支局部覆盖。	控制 Boss 连续出招频率, 是整体压迫的第一阀门。
InitialAttackDelay	默认约 3.0s。	给开场锁定、观察和进入战斗镜头留缓冲。
Distance Range	Close/Mid/Far 由 CPData 与 AttackData 共同决定。	决定招式出现距离, 优先用于修复不合理空挥和过远命中。
PostHitReactRecovery	Normal / Hard / Power 可逐步缩短。	决定玩家命中后能否继续确认输出。
PatientDamageToFill	越低越容易触发 CounterStrike。	控制 Boss 被压制多久后反击; 过低会惩罚正常输出。
DefendDuration	默认防御持续时间可被命中延长。	控制 Boss 格挡站桩时长, 需要和反击阈值一起调。
ReceivedHitCounterLimit	BT 可设置被命中多少次后打破冷却。	让 Boss 在被连续命中时更快回到行动, 避免被无限压制。
MontageSelectionMode	Random、NoImmediateRepeat、ShuffleBag、Sequential。	控制同一 AttackID 下蒙太奇变体的重复度和可读性。

- 如果 Boss 显得过于疯抢, 优先增加 AttackCooldown 或延长 Normal/Hard 的 PostHitReactRecovery。
- 如果玩家能稳定贴身压死 Boss, 优先降低 PatientDamageToFill 或提高 ReceivedHitCounter 打破冷却的频率。
- 如果冲刺、追击或大范围招式不公平, 优先检查 AttackData 距离掩码、MoveToPlayerAttackRange 和 Motion Warping 目标刷新。
- 如果战斗显得随机, 优先收紧 BT 分支条件, 并将核心招式蒙太奇改为 NoImmediateRepeat 或 ShuffleBag。
- 如果战斗过于被动, 优先增强防御、反击、拉距和阶段分支, 而不是简单提高攻击伤害。

6.5 特殊机制和实现方案

6.5.1 AttackIntent 显式调度

当前 AI 的关键不是“CP 算出下一招”，而是“BT 先决定这一支要打什么”。

BTTask_SetAttackIntentCpp 将 AttackID 写入 ABossAIController::CurrentAttackIntent 和 Blackboard 的 AttackID；如果 AttackID 不是 None，还可以要求 CombatData 中存在对应的范围数据，避免树上配置了无效招式。

步骤	入口	职责
1. 分支选择	Behavior Tree	根据阶段、距离、状态、冷却和分支条件选择一个具体 AttackID。
2. 写入意图	BTTask_SetAttackIntentCpp	设置 Controller 当前意图，并同步到 Blackboard AttackID。
3. 移动到位	BTTask_MoveToPlayerAttackRangeCpp	根据意图招式读取 CombatData 距离范围，移动到可执行距离。
4. 执行门控	BTTask_ExecuteAttackCpp / ABossAIController::ExecuteAttack	检查战斗状态、攻击冷却、范围是否合法，再激活能力。
5. 能力播放	GA_BossAttackBase	读取 PendingAttackID，选择 FAttackData、MontageVariants 和 Motion Warping 目标。

6.5.2 CombatPressure

CombatPressureComponent 当前更适合描述为感知与信号层。它 Tick 更新距离、攻击密度、防御倾向、玩家 BV、Patient、Disorder/Calm 等值，并将部分结果通过 Controller 或 BTS_UpdateCP 写入 Blackboard。它可以影响 BT 条件和反击请求，但不应被文档写成普通攻击的唯一或主导选招器。

信号	代码来源	当前用途口径
Distance / DistanceRange	UpdateDistance、GetDistanceRange、UpdateDistanceRangeBlackboard	核心使用。驱动 Close/Mid/Far 判断、移动目标和攻击范围兜底。
Patient / CounterStrike	AccumulatePatientDamage、ShouldTriggerCounterStrike、ResetPatientTimer	核心使用。Boss 被压制到阈值后可请求 CounterStrike，成功后清空请求。
Player BV	UpdatePlayerBV、IsPlayerBVPressured	辅助信号。可供 BT 或调试判断玩家资源压力。

6.5.3 距离和攻击范围兜底

距离系统是当前 AI 最稳定的决策基础。CP 记录当前距离并计算距离段；Controller 每 Tick 同步 DistanceRange、DistanceRangeInt 和 CurrentDistance 到 Blackboard；移动任务按 AttackIntent 查 CombatData 的范围掩码；ExecuteAttack 在真正激活 GA 前再次调用范围检查。这样即使玩家在移动过程中改变位置，Boss 也不会错误距离强行出招。

层级	检查内容	失败结果
BT Decorator	当前距离段是否满足分支条件	该分支不进入，回到其他分支或移动调整。
MoveToPlayerAttack Range	是否已经处于该 AttackID 允许的 Close/Mid/Far	未到位则移动；移动结束仍不到位则失败或重试。
ExecuteAttack	CombatState、冷却、范围掩码、ASC/Ability 是否有效	拒绝执行，避免错误播放蒙太奇。
GA_BossAttackBase	PendingAttackID 是否能找到 CombatData	无数据则无法执行对应招式，便于定位配置问题。

6.5.4 防御、反击与受击恢复

防御和反击是 Boss 从被压制状态夺回主动权的机制。玩家命中 Boss、Boss 处于非攻击状态时，会推进 Patient 或受击计数；达到阈值后可写入 bShouldCounterStrike，BT 或 SelectAndExecuteAttack 的兜底逻辑会优先执行 CounterStrike。防御任务则通过 Block Tag、DefendMontage、ParryDefend 和受击回调，把“被打后立刻反击”和“短时间格挡等待”分成不同表现。

机制	实现节点	设计目的
Idle Defend	BTTask_DefendCpp	Boss 在非攻击状态被连续命中时进入防御，阻止玩家单一连打节奏。
Parry Defend	BTTask_BossParryDefendCpp	按命中侧选择左/右格挡表现，可后撤并在反击请求出现时退出。
CounterStrike	EAttackID::CounterStrike	Patient 或 BB 请求触发，执行后清空 bShouldCounterStrike 并重置 Patient。
PostHitReactRecovery	BTTask_PostHitReactRecoveryCpp	受击后短暂停留并面向玩家，避免状态刚结束就立即移动或出招。

6.5.5 阶段与冷却

BossHealthPhase 每 0.5 秒检查一次血量比例并写入 Blackboard。当前阈值为 Hard 70%、Power 30%。阶段不一定直接改变所有逻辑，但它提供了清晰的 BT 分支入口和调参锚点。攻击冷却由 LastAttackEndTime 与 AttackCooldown 控制，特殊攻击可做冷却豁免；ReceivedHitCounter 相关任务可以在 Boss 被连续命中时打破剩余冷却，让 Boss 更快进入回击分支。

6.6 BT 结构

当前 BT 结构应描述为 “服务同步 + 高优先级状态 + 显式攻击意图 + GAS 执行。本组图采用 “主线路 + 子树展开” 的方式表达：主图只展示 BT 的 Selector 主干，Decorator 和关键条件以标签形式写入节点内部；各子树的内部展开单独成图。

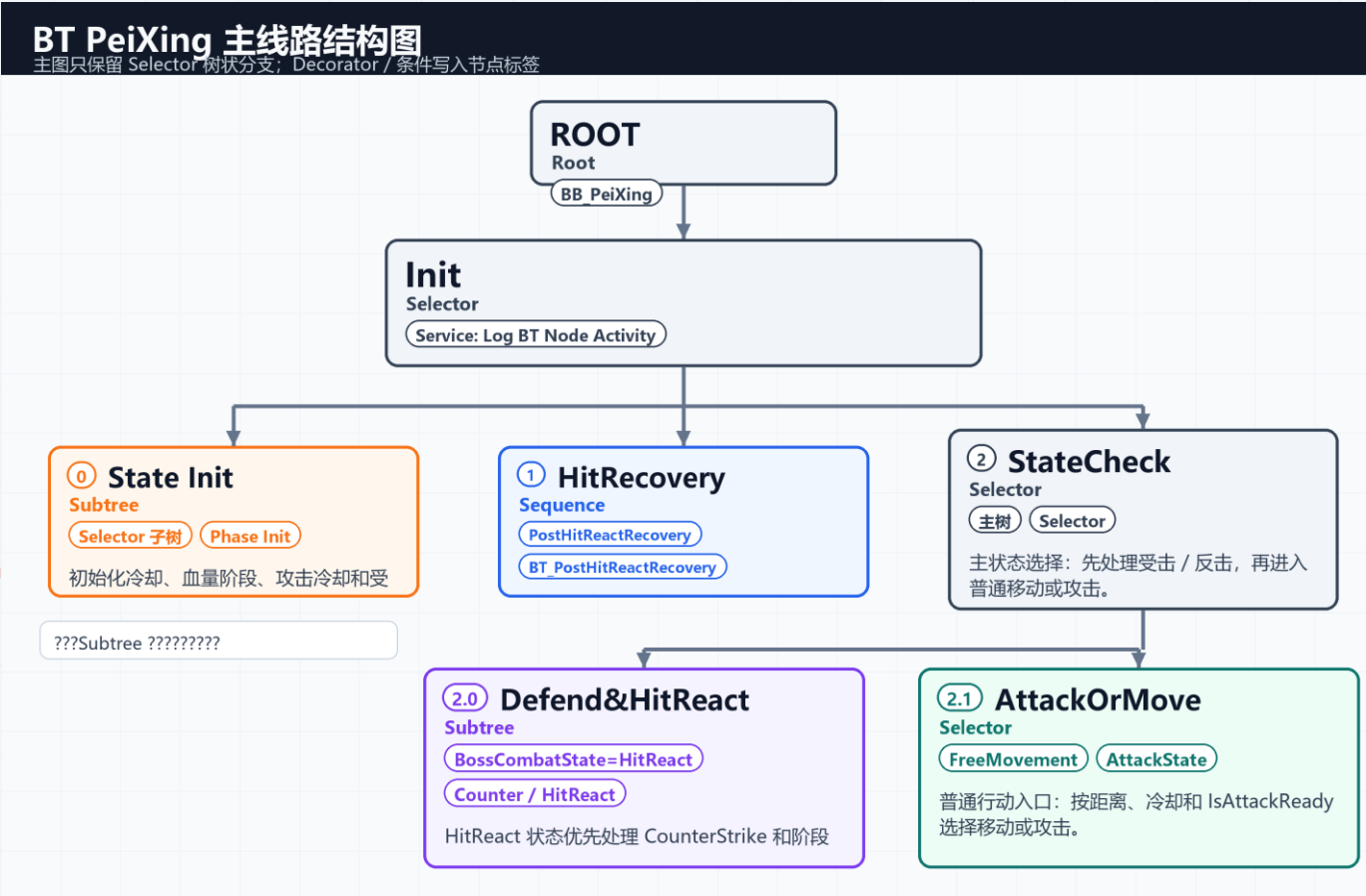


图 6-1 BT_PeiXing 主线路结构：ROOT、Init、StateCheck 与 AttackOrMove 的树状分支

State Init 子树

初始化阶段参数: 冷却锁 -> 阶段初始化 Selector

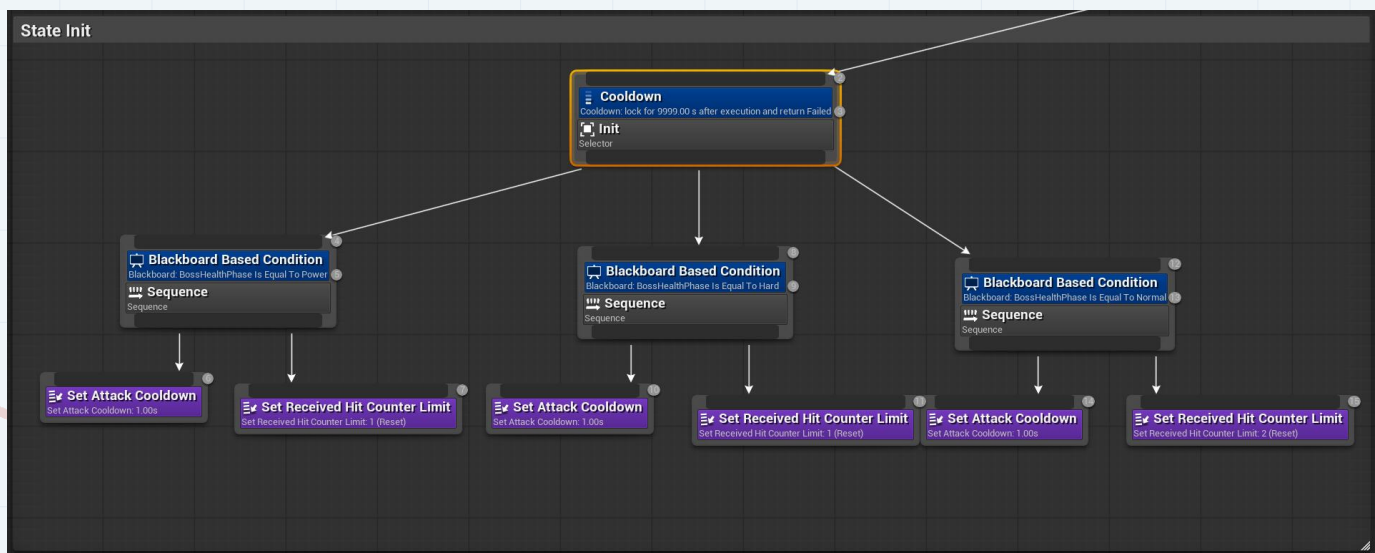
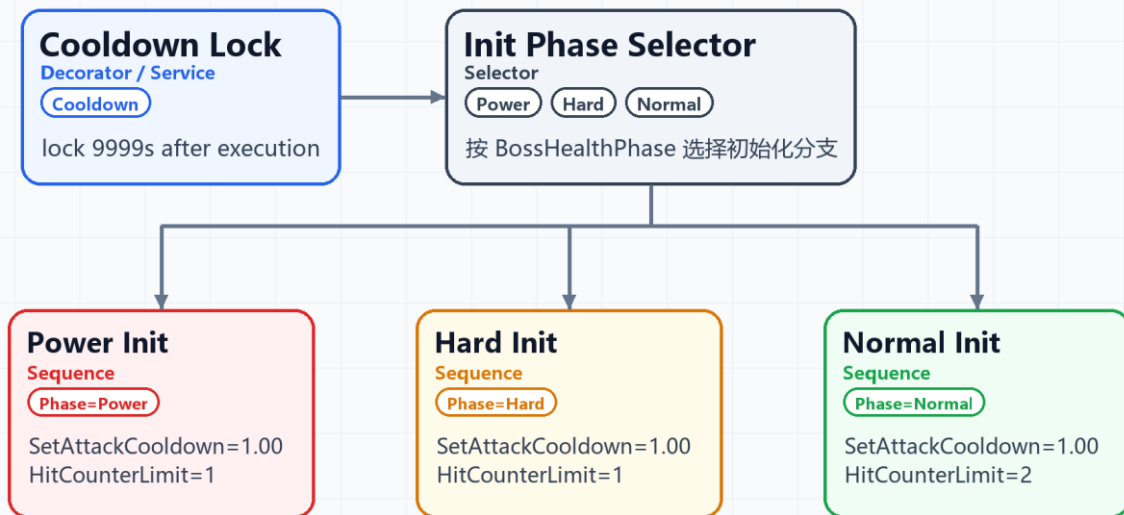


图 6-2 State Init 子树: 冷却锁与 BossHealthPhase 初始化分支

Defend&HitReact 子树

HitReact 状态下的反击与阶段受击分支

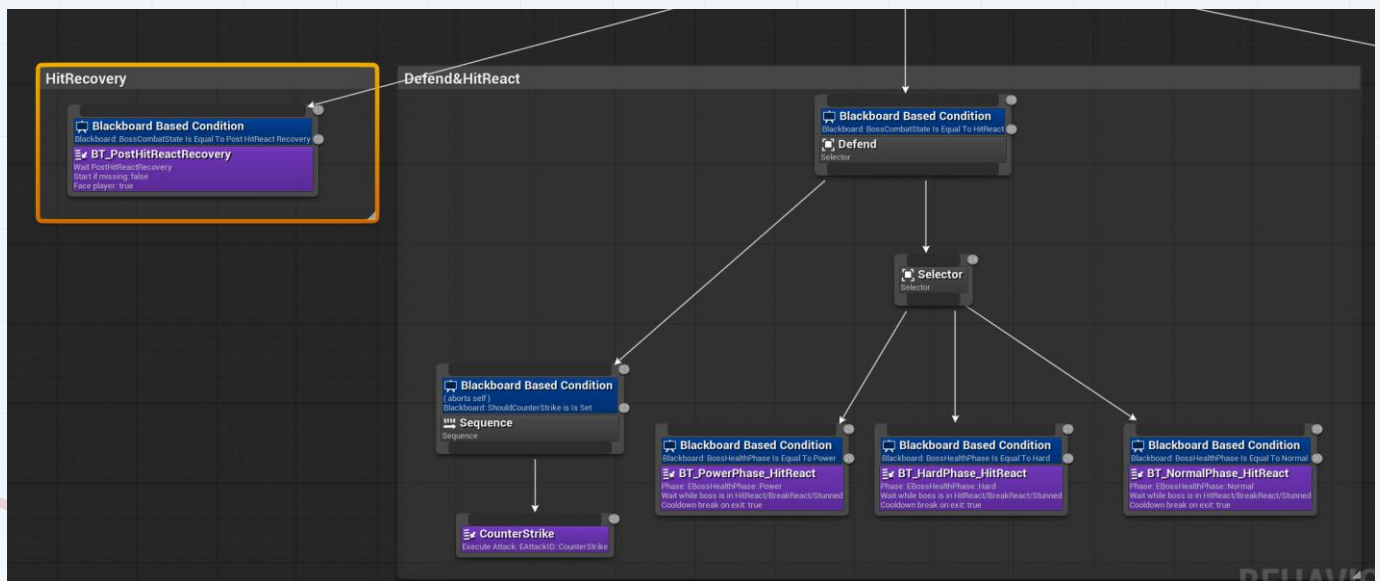


图 6-3 Defend&HitReact 子树：HitReact 状态下的反击与阶段受击分支

FreeMovement 子树

远距离与冷却期站位调整

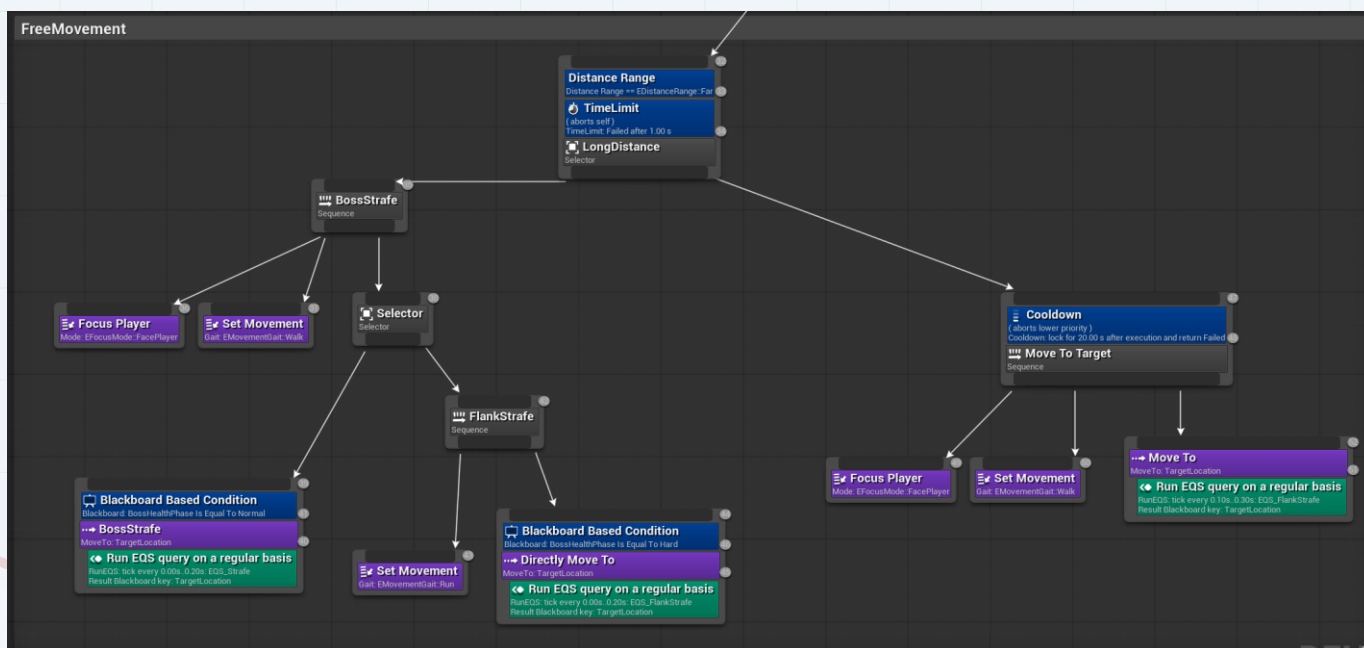
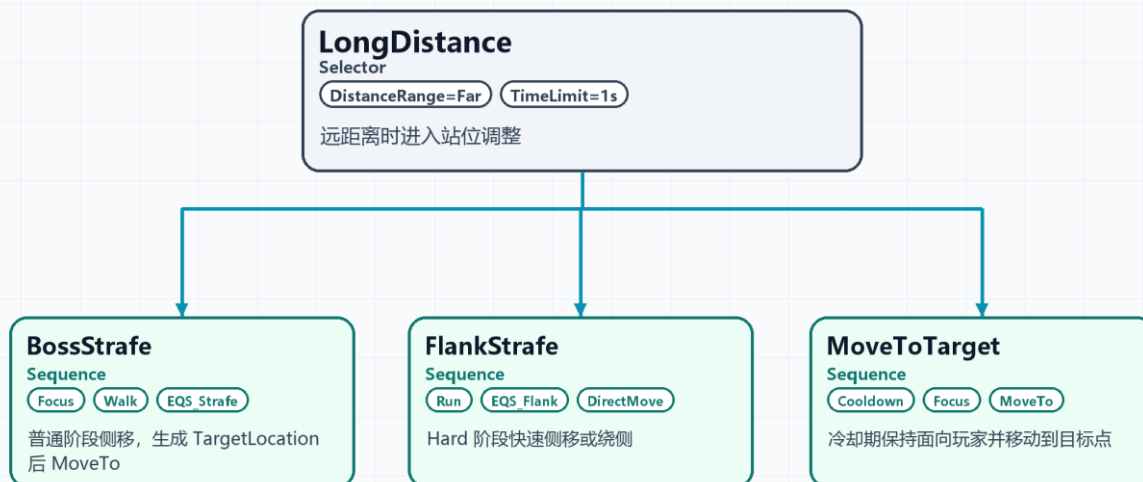


图 6-4 FreeMovement 子树：远距离与冷却期站位调整

AttackState 子树

距离攻击、阶段攻击与冷却动作

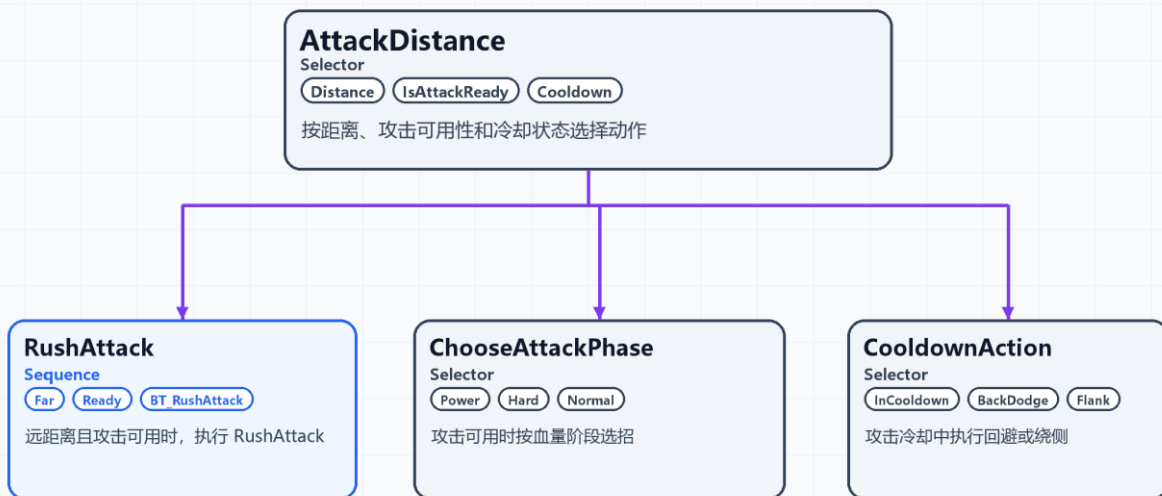


图 6-5 AttackState 子树：距离攻击、阶段攻击与冷却动作

ChooseAttackPhase 子树

按阶段选择攻击，然后进入公共执行 Sequence



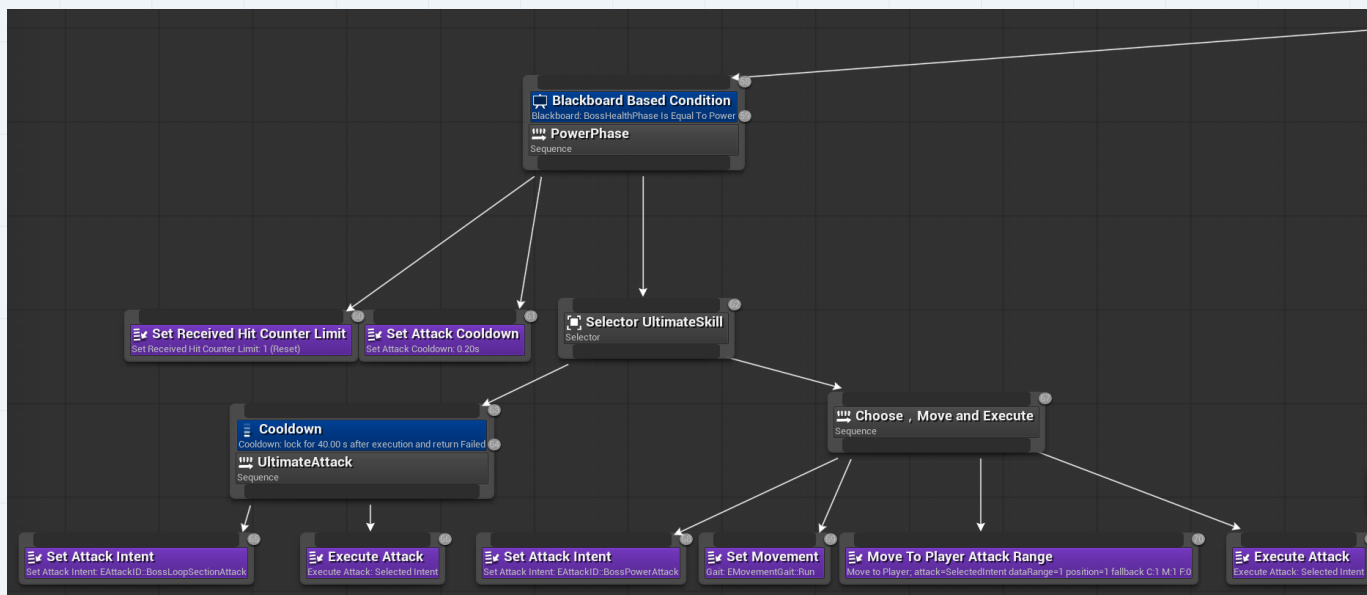


图 6-6 ChooseAttackPhase 子树: Power / Hard / Normal 阶段攻击展开

层级	节点 / 数据	职责
服务层	BTS_UpdateCP、CombatPressure Tick、BossAIController Tick	更新距离、阶段、状态、反击请求和辅助压力信号。
状态层	BossCombatState、HitReact、BreakReact、Stunned、PostHitReactRecovery	死亡、受击和硬直优先，阻止普通移动/攻击穿插。
反击层	bShouldCounterStrike、PatientTimer、CounterStrike	Boss 被压制到阈值时优先反击，打断玩家无成本贴身输出。
防御层	bShouldDefend、BTTask_DefendCpp、BTTask_BossParryDefendCpp	处理被连续命中、前摇被打和格挡循环中的防御反馈。
攻击层	SetAttackIntent、DistanceRange Decorator、MoveToPlayerAttackRange、ExecuteAttack	显式选择 AttackID，移动到位后由 Controller 执行。
调整层	FocusPlayer、SetMovement、EQS、MoveToFreshTargetLocation	冷却或追击间隙重建站位，避免一直追旧位置。

